Actividad 2 - Documento de formulación del proyecto

Daisy Nicol Malagón Ramírez

Luz Celeste Rodríguez Cabarcas

Angela Marcela Salazar Peña

Juan Guillermo Sánchez Restrepo

Ingeniería En Software

Proyecto de Software

Corporación Universitaria Iberoamericana

Tatiana Lizbeth Cabrera Vargas

Bogotá D. C.

Abril, 2026

Tabla De Contenido

Introducción 7

Necesidad Y Problema Del Proyecto 8

Descripción Del Problema 8

Contexto Del Problema 8

Población Afectada 9

Justificación De La Solución Tecnológica 9

Objetivos Del Proyecto 10

Objetivo General 10

Objetivos Específicos 10

Alcance Del Proyecto 10

Inclusiones 10

Exclusiones 12

Limitaciones 12

Recursos Del Proyecto 12

Beneficios Esperados 13

Metodología De Desarrollo 13

Descripción General De Scrum 13

Roles Del Equipo De Trabajo 14

Organización Del Trabajo 14

Herramientas De Gestión 15

Módulos Funcionales Y Tecnologías Previstas 15

Requisitos del sistema 16

Levantamiento de requisitos 16

Requisito Funcionales 16

Requisitos No Funcionales 17

Identificación De Stakeholders Y Usuarios Finales 18

Clasificación De Stakeholders 18

Descripción De Los Usuarios Finales 19

Mapa De Stakeholders De Vecired 19

Historias De Usuario 21

Relación Entre Requisito Funcional E Historias De Usuario 23

Flujos De Solución 24

Diagramas De Flujo Del Sistema 24

Flujos De Interacción Del Usuario 27

Solución Tecnológica Por Implementar 32

Descripción Del Sistema 32

Arquitectura General De La Solución 33

Tecnologías Que Se Utilizarán 36

Beneficios Esperados 36

Modelamiento Del Sistema 37

Diagramas Estructurales 37

Diagrama De Componentes 37

Diagramas De Comportamiento 38

Modelamiento De Base De Datos 42

Segunda Entrega 43

Actualización del MVP 44

Estado del MVP por Historia de Usuario 44

Prototipo funcional del sistema 48

Prototipo de alta fidelidad 48

UI Kit 48

Pruebas de usabilidad 50

Arquitectura del sistema 56

Diagramas C4 56

Documento ADR 60

Registro de Decisiones de Arquitectura 60

Decisiones Detalladas 62

Repositorio del proyecto 64

Modelamiento de Base de Datos — Diagrama Entidad Relación (ERD) 64

Retrospectiva del Sprint 66

¿Qué funcionó bien? 66

¿Qué dificultades se presentaron? 66

¿Qué mejoraremos en el siguiente Sprint? 67

Firmas del Equipo 67

Video 68

Conclusiones 69

Referencias 70

Tabla De Tablas

Tabla 1 16

Tabla 2 17

Tabla 3 23

Tabla 4 44

Tabla 5 45

Tabla 6 45

Tabla 7 46

Tabla 8 46

Tabla 9 47

Tabla 10 47

Tabla 11 50

Tabla 12 50

Tabla 13 51

Tabla 14 52

Tabla 15 53

Tabla 16 54

Tabla 17 60

Tabla De Figuras

Figura 1 19

Figura 2 24

Figura 3 26
Figura 4 27
Figura 5 28
Figura 6 30
Figura 7 32
Figura 8 34
Figura 9 37
Figura 10 37
Figura 11 38
Figura 12 40
Figura 13 41
Figura 14 42
Figura 15 42
Figura 16 48
Figura 17 51
Figura 18 51
Figura 20 52
Figura 21 53
Figura 22 54
Figura 23 57
Figura 24 58
Figura 25 64

Introducción

El presente proyecto da inicio al proceso de formulación, diseño y planeación de VeciRed, una plataforma digital orientada a fortalecer las economías hiperlocales en Medellín mediante la conexión directa entre prestadores de servicios locales y los habitantes de las comunas. La propuesta surge de la necesidad de crear un medio que conecte a los trabajadores independiente con el cliente final, dando seguridad y confianza a través de un sistema de perfiles verificados y reseñas de vecinos reales. Esta solución no solo facilita el acceso al mercado digital para los trabajadores independientes, sino que también optimiza la interacción entre oferta y demanda, promoviendo la formalización laboral y el fortalecimiento de la productividad local.

Para el desarrollo del proyecto se aplicará la metodología ágil Scrum, la cual permite organizar el trabajo en Sprints. Este enfoque facilita la planificación, el desarrollo progresivo y la mejora continua del producto, asegurando que la plataforma evolucione de manera flexible y respondiendo de forma efectiva a los requerimientos de los usuarios. Durante esta fase inicial se establecen las bases conceptuales, técnicas y organizacionales que guiarán el desarrollo de VeciRed, asegurando la coherencia entre los objetivos del proyecto y la solución tecnológica implementada.

VeciRed se enmarca en la línea estratégica “Conexión Local — Economías Hiperlocales”, que busca potenciar la economía local mediante plataformas digitales que conecten productores, comerciantes y consumidores dentro de los barrios. La propuesta tecnológica combina herramientas modernas como Node.js, Express, PostgreSQL, React y sistemas de autenticación seguros con JWT, garantizando la eficiencia, escalabilidad y confiabilidad del sistema. Así, VeciRed es una herramienta para el crecimiento de los prestadores de servicios locales y como un recurso confiable para los habitantes de la comuna, fortaleciendo la circulación de capital dentro del barrio y promoviendo una economía más justa y sostenible.

Necesidad Y Problema Del Proyecto

Descripción Del Problema

El problema central nace de la invisibilidad de los servicios técnicos y manuales tales como la electricidad, costura, pintura, entre otros; ofrecidos por trabajadores que operan de forma independiente desde la informalidad en las 16 comunas de Medellín. Pese a que estas zonas poseen una fuerza laboral técnica robusta, el 92% de los prestadores de servicios en barrios como Aranjuez, La Candelaria y el Poblado actualmente dependen casi exclusivamente del "voz a voz" físico o de métodos de publicidad obsoleta y rudimentaria como los volantes o los letreros en postes teniendo como resultado un alcance limitado y nula trazabilidad de calidad. Esto debido a que no cuentan con un apoyo tecnológico ni una identidad digital profesional.

Contexto Del Problema

La desconexión digital situada en las 16 comunas de Medellin, dificulta el acceso rápido a servicios confiables y obliga a los residentes de las comunas a recurrir a plataformas globales o empresas externas que cobran tarifas más altas por desplazamientos. Mientras tanto, el talento local permanece subempleado por falta de exposición, lo que genera inestabilidad en los ingresos y una disminución en la economía del sector afectando el sector productivo y social.

Según el reporte de Medellín Cómo Vamos (Actualizado a 2025-2026) y los datos del Observatorio de Desarrollo Económico de Medellín, la tasa de informalidad laboral en la ciudad se sitúa en un 41.8%. Sin embargo, en los estratos 1, 2 y 3, más del 60% de los ingresos de los hogares provienen de actividades independientes o de "oficios". La carencia de un sistema de reputación digital local impide que estos trabajadores escalen sus ingresos, manteniendo un ciclo de economía de subsistencia en lugar de una economía hiperlocal escalable.

Población Afectada

Los usuarios involucrados en esta situación son los prestadores de servicios informales, tanto hombres como mujeres de las comunas de Medellín con habilidades técnicas (electricistas, modistas, pintores, etc.) que no tienen acceso a plataformas de marketing digital y los habitantes de las comunas, que son todos aquellos vecinos que requieren de algún servicio y/o reparaciones; y buscan seguridad, precios justos, cercanía y rapidez, pero que a hoy, no cuentan con una herramienta que les permita encontrar fácilmente a estos trabajadores locales.

Justificación De La Solución Tecnológica

La implementación de VeciRed como plataforma digital se justifica en la necesidad de digitalizar la confianza comunitaria para mejorar las economías hiperlocales de Medellín. Esta solución permite transformar la reputación informal en un activo digital medible mediante un sistema de perfiles verificados y reseñas de vecinos reales, eliminando las barreras de entrada al mercado digital para el trabajador independiente, aumenta la confiabilidad y disminuye los costos para el usuario final.

La propuesta de valor de VeciRed: "Talento de confianza, a la vuelta de tu casa", se centra en el empoderamiento del prestador de servicios local, brindándole una vitrina digital profesional y autogestionada. Para los habitantes de la comuna, la plataforma asegura agilidad al verificar la cercanía y reducir los tiempos de espera; seguridad al permitir comprobar la experiencia, evidencias de trabajos y reseñas; y equidad al fomentar una economía circular en la que el capital fluye dentro del mismo barrio. En definitiva, esta solución tecnológica no solo optimiza el encuentro entre oferta y demanda, sino que también impulsa la formalización social y fortalece el tejido productivo en los sectores con mayores índices de informalidad de la ciudad.

Objetivos Del Proyecto

Objetivo General

Desarrollar una plataforma web para conectar prestadores de servicios técnicos con habitantes de las comunas de Medellín aumentando la confianza comunitaria y la economía hiperlocal.

Objetivos Específicos

- Estructurar un sistema de información que permita la autogestión de perfiles de usuarios, la clasificación y el registro histórico de los servicios adquiridos.
- Diseñar una interfaz digital intuitiva que facilite la localización de servicios mediante filtros geográficos y categorías de especialidad.
- Implementar un módulo de seguridad y control de acceso que garantice la privacidad de los datos personales y la autenticidad de cada interacción dentro del ecosistema digital.
- Integrar un mecanismo de reputación basado en valoraciones y comentarios verificados para fortalecer el vínculo de confianza entre los actores locales.

Alcance Del Proyecto

El proyecto VeciRed integrará el diseño, desarrollo y despliegue de una aplicación web orientada a la conexión hiperlocal en las comunas de Medellín. El sistema permitirá la gestión de dos tipos de perfiles: el Prestador, quien podrá publicar sus servicios técnicos (electricidad, costura, pintura, etc.), gestionar una galería de evidencias de su trabajo y definir su zona de cobertura; y el Vecino (Cliente), quien tendrá la posibilidad de buscar, filtrar y visualizar perfiles de trabajadores cercanos a su ubicación. La solución incluirá un módulo de autenticación segura y un sistema de reputación basado en valoraciones cualitativas y cuantitativas, garantizando que la interacción sea transparente y basada en la confianza comunitaria.

Inclusiones

El sistema incluirá las siguientes funcionalidades principales:

- Gestión de Identidad: inicio de sesión y registro con roles diferenciados (Cliente y Prestador de Servicio) y protección de datos mediante cifrado.
- Buscador: con filtros avanzados por categorías de oficio y ubicación geográfica dentro de las 16 comunas de Medellín.
- Dashboard del Prestador: Espacio personalizado para el prestador donde ingresara sus datos incluyendo zonas de cobertura, descripción, fotos de trabajos realizados y muro de recomendaciones.
- Sistema de Reputación: Mecanismo de calificación por estrellas y comentarios verificados para cada servicio prestado.

Asimismo, para garantizar el correcto funcionamiento y la gestión de los contenidos en VeciRed, el sistema contará con un nivel de acceso restringido para los administradores del proyecto. Este módulo no será visible para el público general y funcionará mediante la asignación de roles y permisos gestionados desde la base de datos y validados a través de rutas protegidas en el servidor. El sistema contará con las siguientes funcionalidades como administrador:

- Moderación de Contenido: Capacidad para supervisar, ocultar o eliminar reseñas y comentarios que infrinjan las normas de convivencia comunitaria.
- Gestión de Usuarios: Facultad para suspender perfiles de prestadores o vecinos en caso de reportes por malas prácticas o fraude.
- Administración de Categorías: Control sobre el catálogo de servicios (ej. añadir "Cuidado de adultos mayores" o "Plomería").
- Monitoreo de Datos: Acceso a una interfaz de visualización de métricas básicas, como el número de usuarios activos por comuna y los servicios más demandados.

Exclusiones

Con el fin de asegurar la estabilidad del despliegue y teniendo en cuenta el enfoque del proyecto, no se incluirán los siguientes aspectos:

- Pasarelas de Pago: El sistema no procesará transacciones monetarias internas (pagos con tarjeta, PSE, etc.). El acuerdo económico y el pago del servicio se realizarán de forma externa y directa entre el vecino y el prestador.
- Chat en Tiempo Real: La comunicación se facilitará mediante enlaces directos a canales externos (como WhatsApp o llamadas telefónicas), evitando la complejidad de mantenimiento de un servidor de mensajería instantánea.
- Geolocalización por GPS en Tiempo Real: La ubicación se gestionará mediante la selección manual de comunas y barrios por parte del usuario, evitando el uso de APIs de mapas pagas o de alta complejidad técnica.

Limitaciones

El desarrollo del proyecto está sujeto a las siguientes condiciones:

- Presupuesto Cero: Solo se permite el uso de capas gratuitas (Free Tiers) de servicios en la nube.
- Seguridad y Privacidad de Datos: No podemos manejar datos sensibles (como fotos de cédulas o documentos legales) dado que no se cuenta con protocolos de ciberseguridad avanzada ni validación de identidad, por lo que el sistema se basara en la reputación de los vecinos.
- Limitaciones de los Servicios Gratuitos: las herramientas en su versión gratuita tienen límites de espacio o uso.
- Dependencia de Terceros (APIs Externas)

Recursos Del Proyecto

Para la ejecución exitosa de la solución, se han identificado los siguientes recursos:

- Talento Humano: Equipo de desarrollo conformado por cuatro (4) estudiantes de Ingeniería de Software. La docente de la asignatura actuará como Stakeholder y Consultora Externa, validando el cumplimiento de los requisitos técnicos en cada entrega.
- Recursos Tecnológicos
    - Hardware: Equipos de cómputo personales con capacidad de procesamiento para entornos de desarrollo local.
    - Conectividad: Acceso a internet de banda ancha para la sincronización de repositorios y pruebas en la nube.
    - Software y Herramientas: Visual Studio Code, Postman (pruebas de API), pgAdmin (gestión de base de datos) y Modelos de Lenguaje Extensos (IA) como apoyo en la optimización de código y documentación.

Beneficios Esperados

La implementación de VeciRed generará una vitrina digital gratuita para el talento informal, aumentando sus oportunidades laborales y estabilidad económica. Para la comunidad, el beneficio principal radica en la reducción del tiempo y costo de búsqueda de servicios confiables, fortaleciendo el tejido social y la economía circular en los barrios. Tecnológicamente, el proyecto demostrará la viabilidad de soluciones de alto impacto social mediante el uso eficiente de herramientas de desarrollo gratuitas y arquitecturas de software modernas.

Metodología De Desarrollo

Descripción General De Scrum

Este proyecto sera desarrollado bajo el marco de trabajo Scrum, una metodología ágil diseñada para optimizar la entrega de valor mediante ciclos iterativos, incrementales y con la disminución de errores. Este enfoque permitirá al equipo inspeccionar el progreso del software

de manera constante, adaptando las funcionalidades según la retroalimentación obtenida y garantizando un producto funcional.

Roles Del Equipo De Trabajo

El equipo está constituido por cuatro desarrolladores autogestionados que participan activamente en la codificación del sistema, asumiendo adicionalmente responsabilidades específicas para la coordinación del proyecto:

- Angela (Product Owner & Developer): recopila los requerimientos de funcionalidad del sistema. Su labor técnica se centra en el diseño y programación de interfaces en React, asegurando que la plataforma sea intuitiva para el usuario final.
- Juan (Scrum Master & Developer): programara y facilitara las Ceremonias de Scrum y gestiona el tablero en Jira. Técnicamente, lidera la configuración del servidor y la API en Node.js, garantizando la fluidez de los datos.
- Daisy (UI/UX Developer): se encargará de la arquitectura visual y adaptabilidad del sitio web. Su responsabilidad diseñar y asegurar que la plataforma sea funcional.
- Luz Celeste Rodríguez Cabarcas (Backend & DevOps Developer): Responsable de la persistencia de datos en PostgreSQL, la implementación de seguridad mediante JWT y el despliegue continuo de la aplicación en servidores de producción.

Organización Del Trabajo

La ejecución del proyecto VeciRed se orientará por Sprints, con una duración de dos semanas cada uno. El flujo de trabajo iniciará con el diseño de la arquitectura del sistema y culminará con el despliegue final en producción, cuya fecha límite de entrega es el 7 de junio de 2026.

Para asegurar la sincronización del equipo y la calidad del código, se implementarán las cuatro ceremonias oficiales de Scrum, adaptando su duración al ciclo de dos semanas del proyecto:

- Sprint Planning: se estima una duración de 4 horas por Sprint
- Daily Scrum: tendra una duración de 15min respondiendo los tres pilares del progreso
- Sprint Review: se estima una duración de 2 horas
- Sprint Retrospective: se proyecta una duración de 90 minutos

Herramientas De Gestión

La administración del proyecto se apoyará en un ecosistema digital que garantice la trazabilidad y la integridad del software

- Herramienta de Gestión: Se utilizará Jira Software para la creación del Product Backlog, definición de Historias de Usuario y seguimiento de tareas en el tablero Kanban.
- Control de Versiones: Uso estricto de GitHub bajo la estrategia de ramificación Git Flow para asegurar la integridad del código.

Módulos Funcionales Y Tecnologías Previstas

Para el desarrollo de VeciRed, se implementará una arquitectura desacoplada utilizando las líneas de stack tecnológico base: Node.js, Express, React, Base de Datos, Figma, DevOps. Los módulos se definen de la siguiente manera:

- Módulo de Autenticación y Seguridad: Gestión de registros e inicios de sesión mediante JWT (JSON Web Tokens) y cifrado de contraseñas
- Módulo de Perfiles y Portafolio: Visualización dinámica de datos del prestador, incluyendo una galería de imágenes gestionada a través de Cloudinary
- Módulo de Búsqueda y Filtros: Motor de consulta que permite segmentar servicios por las 16 comunas de Medellín y categorías de oficio.
- Módulo de Reputación: Sistema de retroalimentación cualitativa y cuantitativa para la validación de servicios.
- Módulo de Despliegue (DevOps): Integración continua y despliegue automático utilizando GitHub Actions, Vercel (Frontend) y Render (Backend/DB).

Requisitos del sistema

Para el desarrollo de la plataforma VeciRed, se definieron los requisitos del sistema a partir del análisis de la problemática identificada, teniendo en cuenta las necesidades reales de los usuarios dentro de la economía local.

Levantamiento de requisitos

El levantamiento de requisitos se identificó, y se analizó las necesidades reales de los usuarios.

- Entrevistas
- Observación del entorno social
- Historias de usuario: Se utilizaron para representar los requisitos funcionales del sistema y de los usuarios.

Requisito Funcionales

Tabla 1

Descripción de requisitos funcionales

Código	Nombre	Descripción	Usuarios	Prioridad
RQF001	Registro de usuarios	El sistema debe permitir registrarse como vecino o prestador de servicio	Todos	Alta
RQF002	Inicio de sesión	El sistema debe permitir autenticarse mediante correo y contraseña	Todos	Alta
RQF003	Gestión de perfil	El prestador puede crear y editar su perfil	Prestador	Alta

RQF004	Búsqueda de servicios	El usuario puede buscar servicios por categoría	Vecino	Alta
RQF005	Filtro por ubicación	El usuario puede filtrar por comuna o zona	Vecino	Media
RQF006	Visualización de perfiles	El sistema muestra información del prestador	Vecino	Alta
RQF007	Calificación de servicios	El usuario puede calificar y comentar	Vecino	Alta
RQF008	Contacto con prestador	El usuario puede contactar al prestador	Vecino	Alta
RQF009	Administración del sistema	El sistema debe permitir gestionar usuarios y moderar contenido	Administrador	Media

Nota: La tabla detalla los requerimientos funcionales del proyecto VeciRed. Elaboración propia.

Requisitos No Funcionales

Tabla 2

Descripción de requisitos no funcionales

Código	Nombre	Descripción	Usuario	Prioridad
RQNF001	Seguridad	Los datos deben almacenarse cifrados para evitar accesos no autorizado	Todos	Alta
RQNF002	Rendimiento	El sistema debe responder en menos de 3 segundos	Todos	Alta

RQNF003	Usabilidad	La plataforma debe ser fácil de usar para cualquier usuario	Todos	Media
RQNF004	Disponibilidad	El sistema debe estar disponible al menos el 99% del tiempo	Todos	Alta
RQNF005	Escalabilidad	El sistema debe soportar múltiples usuarios sin fallos	Sistema	Media
RQNF006	Integridad de datos	La información debe mantenerse consistente y segura	Sistema	Alta

Nota: La tabla detalla los requerimientos no funcionales del proyecto VeciRed. Elaboración propia.

Identificación De Stakeholders Y Usuarios Finales

Clasificación De Stakeholders

- Stakeholders Internos:
  - Equipo Scrum: conformado por Juan, Daisy, Angela, Luz Celeste; como desarrolladores, tienen la máxima influencia sobre el código y un interés total en el cumplimiento de los objetivos académicos y técnicos.
- Stakeholders Externos
  - Ingeniera Tatiana Cabrera: Actúa como el cliente principal y evaluador. Su influencia es alta, ya que define los requisitos y la aprobación final.
  - Proveedores de Infraestructura (Vercel, Render, Cloudinary): Proporcionan los servicios de nube gratuitos. Su interés en este proyecto específico es bajo, pero su influencia es alta (si el servicio cae, la app no funciona).

- Comunidad y Entes Locales de Medellín: Organizaciones comunales que están interesadas en la dinamización de la economía local.

Descripción De Los Usuarios Finales

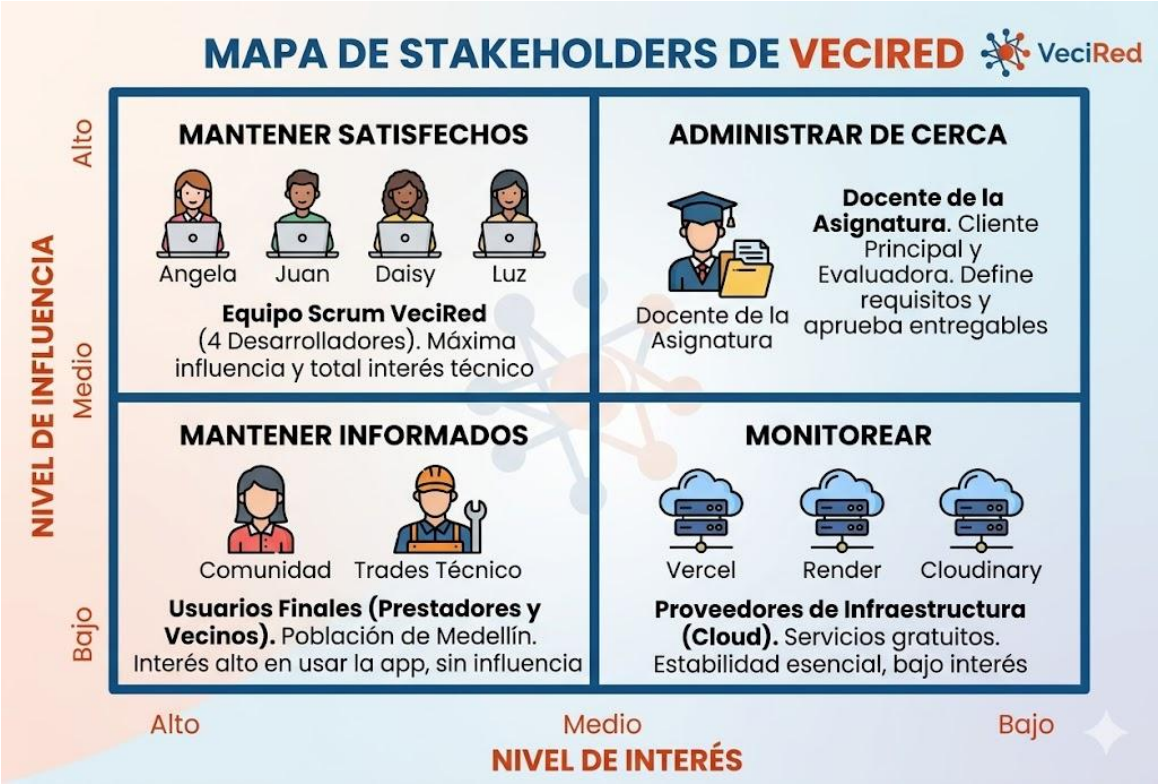
- El Prestador (Trabajador informal): es el usuario activo que gestiona su identidad digital. Su rol, puede crear su perfil, describir su oficio (electricista, modista, etc.), subir fotos de sus trabajos y definir su zona de cobertura en las comunas.
- El Vecino (Cliente local): es el usuario consultor de la plataforma y el que realizara las búsquedas geolocalizadas, contactara al prestador y validara la calidad del servicio mediante el sistema de reputación.

Mapa De Stakeholders De Vecired

Basado en el modelo de gestión propuesto, los actores se ubican en los siguientes cuadrantes:

Figura 1

Matriz de Stakeholders para el proyecto VeciRed



Nota: Elaborado mediante el modelo de lenguaje de IA Gemini 3 Flash (2026) a partir del prompt: "Genera una imagen de la matriz de stakeholders para el proyecto VeciRed, siguiendo el modelo de HubSpot relacionado con los cuadrantes: Mantener satisfechos, Administrar de cerca, Mantener informados y Monitorear, incluyendo al equipo Scrum, la docente y proveedores y teniendo en cuenta los siguientes actores: prestadores de servicio, vecinos, equipo Scrum: conformado por Juan, Daisy, Angela, Luz Celeste, docente de la asignatura, proveedores de Infraestructura (Vercel, Render, Cloudinary) y la comunidad y los entes Locales de Medellín que se interesan en la economia "

Historias De Usuario

HU01

Actor: Vecino

Necesidad: Registro en la plataforma

Finalidad: Acceder a los servicios disponibles en su comunidad

Criterios de aceptación:

- El usuario puede registrarse con nombre, correo y contraseña
- El sistema valida que el correo no esté previamente registrado
- El sistema confirma el registro exitoso

HU02

Actor: Prestador de servicios

Necesidad: Crear y gestionar su perfil

Finalidad: Dar a conocer sus servicios y generar confianza a los usuarios

Criterios de aceptación:

- El prestador puede ingresar información de sus servicios
- Puede definir su zona de cobertura
- Puede actualizar su perfil en cualquier momento

HU03

Actor: Vecino

Necesidad: Buscar servicios por categoría

Finalidad: Encuentra rápidamente el servicio que necesita

Criterios de aceptación:

- El usuario puede seleccionar una categoría
- El sistema muestra servicios relacionados

- La búsqueda arroja resultados correctos

HU04

Actor: Vecino

Necesidad: Filtra los resultados por ubicación

Finalidad: Identifica prestadores cercanos a la zona

Criterios de aceptación:

- El usuario puede seleccionar una zona
- El sistema muestra solo prestadores de esa ubicación
- El filtro se aplica correctamente

HU05

Actor: Vecino

Necesidad: Visualiza calificaciones y comentarios de otros usuarios

Finalidad: Selecciona un prestador confiable

Criterios de aceptación:

- El sistema muestra la puntuación del prestador
- Se visualizan comentarios de otros usuarios
- La información es visible antes del contacto

HU06

Actor: Vecino

Necesidad: Contactar al prestador

Finalidad: Coordina la prestación del servicio

Criterios de aceptación:

- El sistema muestra un medio de contacto (Teléfono y WhatsApp)
- El usuario puede acceder fácilmente al contacto
- La información es clara y visible

HU07

Actor: Administrador

Necesidad: Gestiona usuarios y modera comentarios

Finalidad: Garantiza la calidad y seguridad de la plataforma

Criterios de aceptación:

- El administrador puede eliminar comentarios inapropiados
- Puede gestionar usuarios registrados
- Puede supervisar todas las actividades del sistema

Relación Entre Requisito Funcional E Historias De Usuario

Tabla 3

Detalle de la relación entre requisito funcional e historias de usuario

Código	Nombre	HU	Relación
RQF001	Registro de usuarios	HU01 – Registro en la plataforma	Necesidad del usuario de registrarse
RQF002	Inicio de sesión	HU01	Permite al usuario acceder al sistema luego del registro
RQF003	Gestión de perfil	HU02 – Gestión de perfil del prestador	Crea y administra la información del prestador
RQF004	Búsqueda de servicios	HU03 – Búsqueda por categoría	Refleja la función de búsqueda
RQF005	Filtro por ubicación	HU04 – Filtro por zona	Permite al usuario encontrar servicios más cercanos según su ubicación

RQF006	Visualización de perfiles	HU05 – Ver calificaciones	Permite visualizar información del prestador, incluyendo las valoraciones y calificaciones
RQF007	Calificación de servicios	HU05 – Ver calificaciones	Relaciona selección y consulta de calificaciones
RQF008	Contacto con prestador	HU06 – Contactar prestador	Permite la interacción directa entre usuario y prestador
RQF009	Administración del sistema	HU07 – Gestión administrativa	Permite controlar usuarios y contenido dentro de la plataforma

Nota: La tabla detalla la relación entre requisito funcional e historias de usuario del proyecto VeciRed.

Elaboración propia.

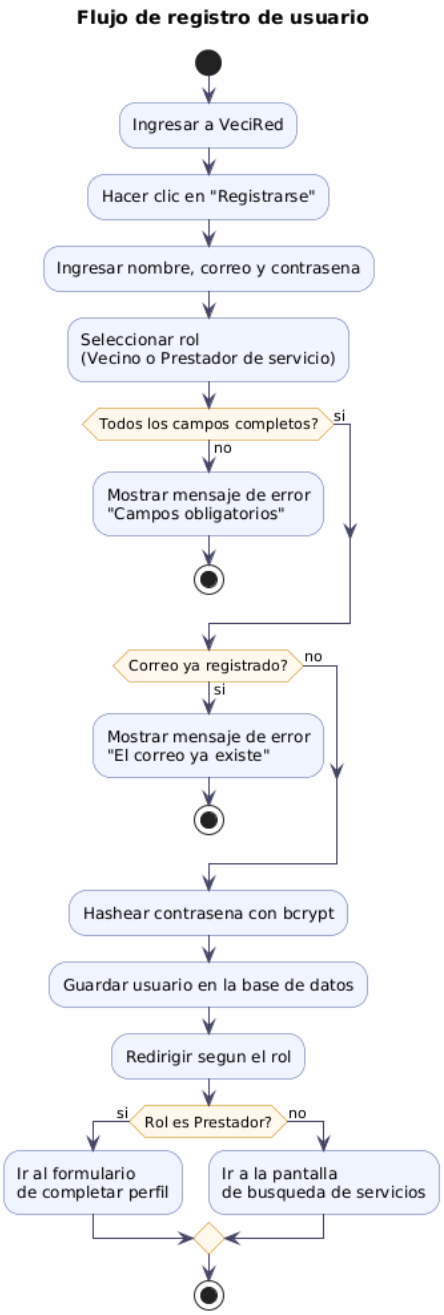
Flujos De Solución

Diagramas De Flujo Del Sistema

El sistema de VeciRed se organiza en flujos que representan la interacción entre los usuarios (prestadores de servicios y habitantes de la comuna) y la plataforma.

Figura 2

Flujo de solución de registro al sistema del usuario

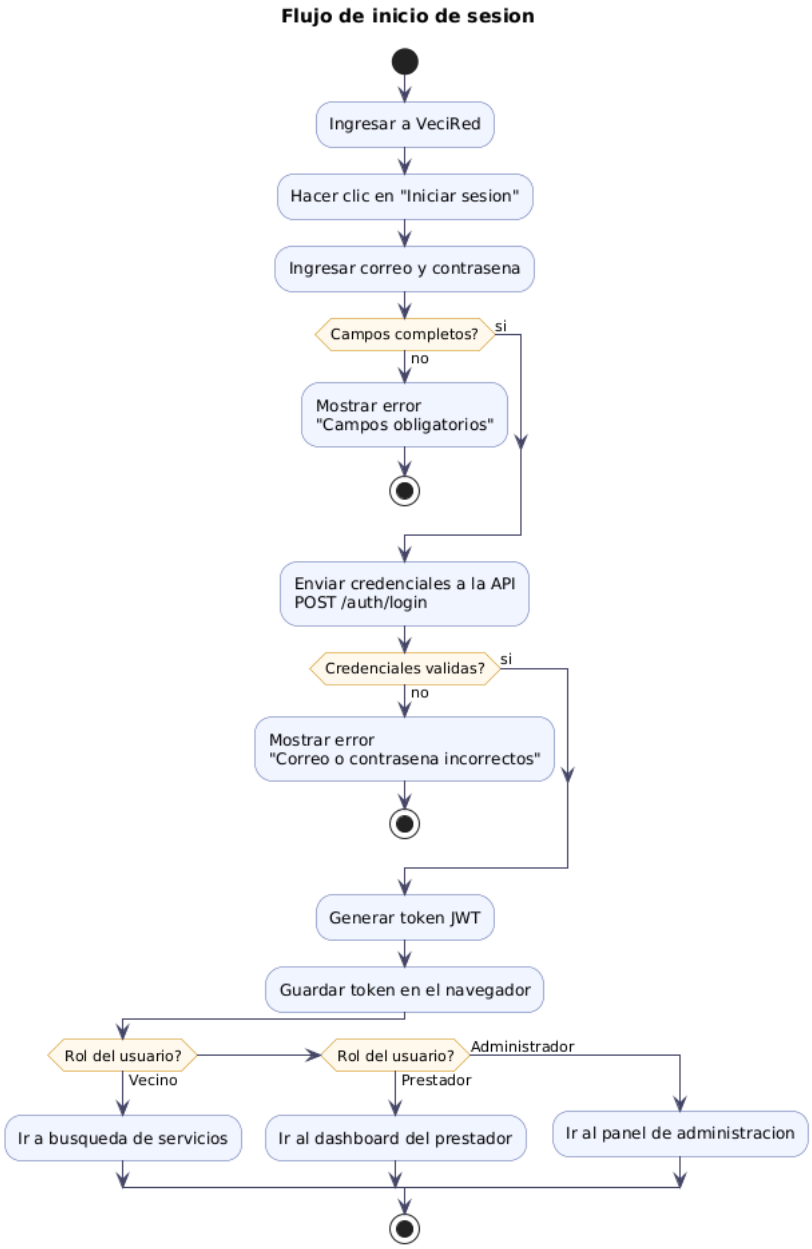


Nota: Este flujo describe el registro, el sistema solicita el ingreso de datos personales, los cuales son validados para garantizar su integridad y consistencia. Si la información es correcta, se procede a la creación del perfil; en caso contrario, se genera un mensaje de error que permite al usuario corregir los datos.

Elaboración propia.

Figura 3

Flujo de inicio de sesión



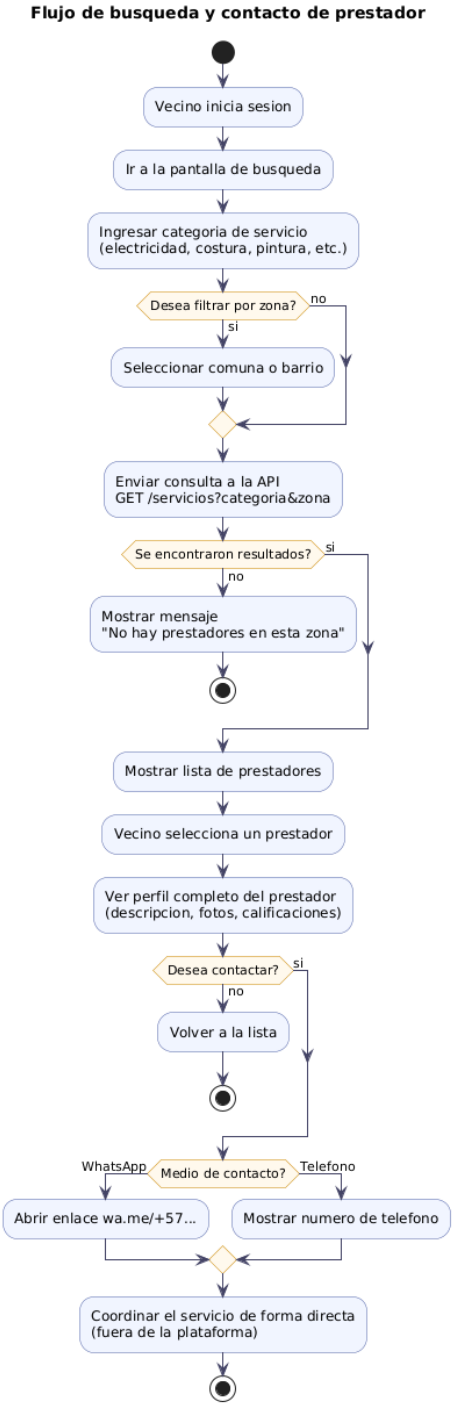
Nota: Este flujo describe el proceso mediante el cual los usuarios acceden a la plataforma, cuando ya están registrados y van a iniciar sesión, el sistema valida las credenciales ingresadas y permite el acceso únicamente si estas son correctas, garantizando la seguridad del sistema.

Elaboración propia.

Flujos De Interacción Del Usuario

Figura 4

Flujo de búsqueda y contacto con el prestador



Nota: El flujo representa el proceso en el que un busca y contacta a un prestador de servicios en la plataforma. Inicia con la opción de filtrar por zona, donde el usuario puede seleccionar una comuna o barrio, y el sistema envía la consulta a la API para obtener resultados según los filtros indicados.

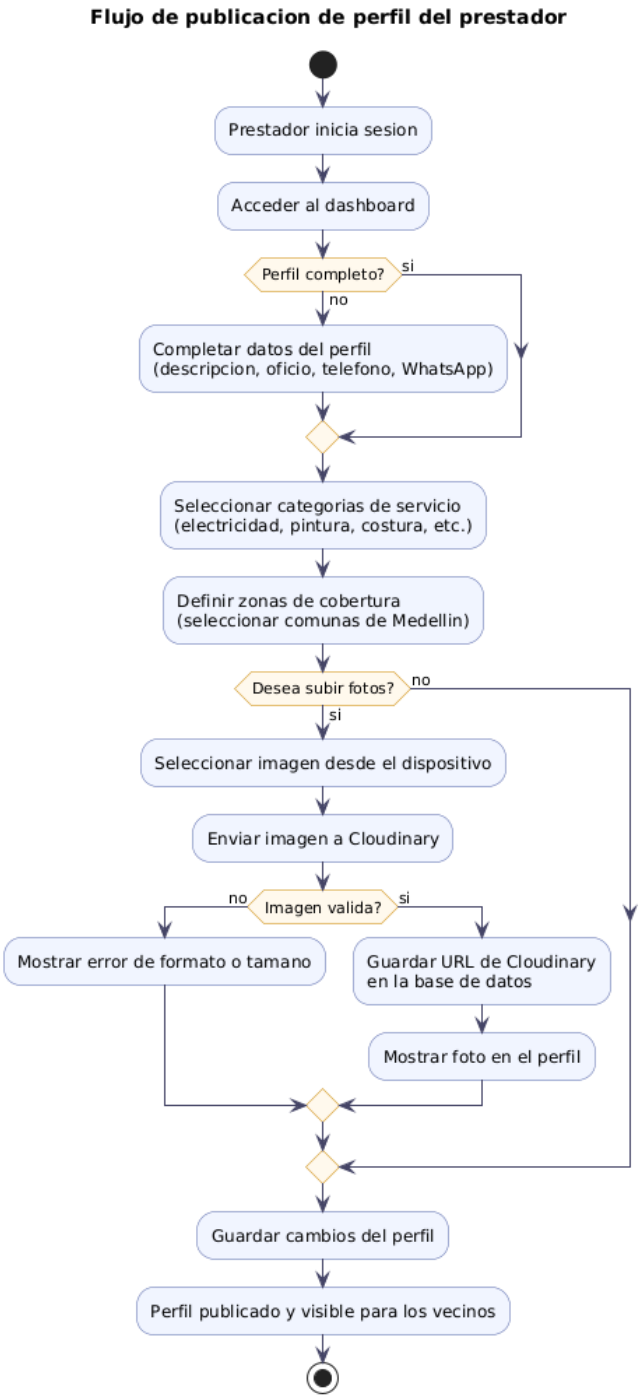
Si no se encuentran prestadores, se muestra un mensaje informativo. En caso contrario, se presenta una lista de opciones, permitiendo al usuario seleccionar y visualizar el perfil completo, para que luego, el usuario decide si desea contactarlo o volver a la lista.

Si decide continuar, el sistema ofrece los medios de contacto, facilitando la coordinación directa del servicio fuera de la plataforma.

Elaboración propia.

Figura 5

Flujo de publicación de perfil del prestador



Nota: El flujo inicia cuando el prestador de servicios inicia sesión y accede al dashboard. Luego, el sistema verifica si el perfil está completo; en caso contrario, el usuario debe completar sus datos personales, incluyendo descripción, oficio y medios de contacto. Una vez

completado, selecciona las categorías de servicio que ofrece y define en que zonas que presta sus servicios.

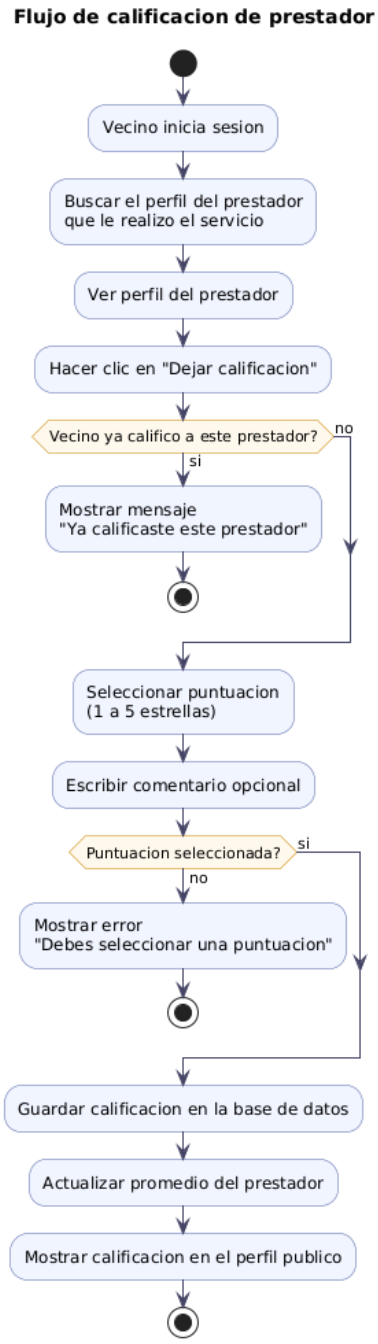
Al terminar esto, el usuario puede optar por subir fotos. Si decide hacerlo, selecciona imágenes desde su dispositivo y estas se envían. El sistema valida las imágenes; si hay errores, se muestra un mensaje. Si son válidas, se guarda la URL en la base de datos y se muestra la foto en el perfil.

Finalmente, el usuario guarda los cambios, dejando su perfil publicado y visible para los vecinos.

Elaboración propia.

Figura 6

Flujo de calificación del prestador



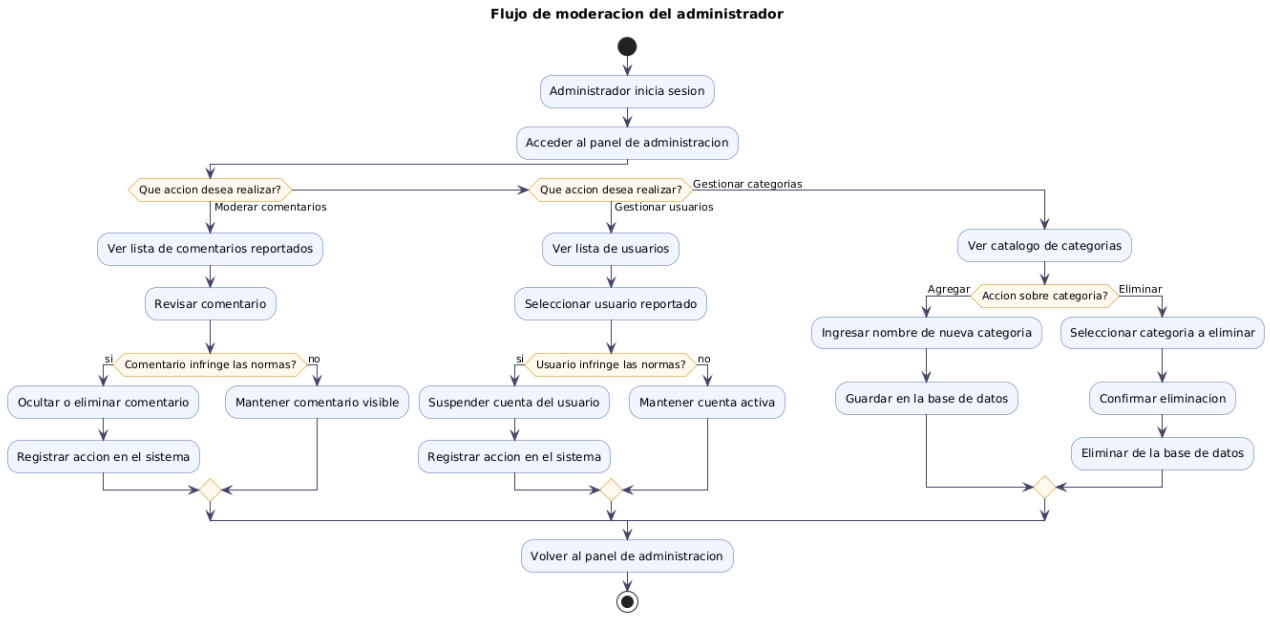
Nota: El flujo señala el proceso mediante el cual un usuario (cliente) califica a un prestador de servicios. El sistema valida si el usuario ya ha calificado previamente; en caso afirmativo, muestra un mensaje informativo. Si no, permite seleccionar una puntuación y agregar un comentario. Luego, verifica que la puntuación haya sido ingresada; si falta, se

muestra un error. Finalmente, la calificación se guarda en la base de datos, se actualiza el promedio del prestador y se refleja en su perfil.

Elaboración propia.

Figura 7

Flujo de moderación del administrador



Nota: El flujo representa el proceso de gestión del administrador dentro del sistema, iniciando con el acceso al panel de administración luego de iniciar sesión. Luego, el administrador puede elegir entre tres acciones principales: revisar comentarios reportados, gestionar usuarios o administrar categorías. El sistema permite evaluar la información y tomar decisiones (eliminar contenido, suspender usuarios o modificar categorías), registrando todas las acciones realizadas. Finalmente, el administrador regresa al panel principal, asegurando la información manejada en el sistema. Elaboración propia.

Solución Tecnológica Por Implementar

Descripción Del Sistema

La plataforma web VeciRed es un medio para conectar a los emprendedores locales con los habitantes de Medellín. El sistema permite la creación y gestión de perfiles verificados,

publicación y búsqueda de servicios, registro histórico de interacciones, y un sistema de reputación basado en reseñas y valoraciones verificadas. Su enfoque principal es fortalecer la confianza comunitaria y formalizar las interacciones económicas locales.

Arquitectura General De La Solución

El sistema está diseñado bajo una arquitectura de tres capas claramente separadas, lo que permite escalabilidad, mantenibilidad y una correcta distribución de responsabilidades entre frontend, backend y base de datos.

Capa 1: Presentación

En esta capa interactúan los tres tipos de usuarios: vecino, prestador y administrador. Está desarrollada con React y es responsable de la interfaz gráfica y la experiencia del usuario.

Se compone de:

Páginas: login, búsqueda, perfil y dashboard, donde los usuarios realizan sus acciones principales.

Componentes reutilizables: como navbar, tarjetas de servicios y sistema de calificaciones.

Contexto global: maneja el estado de autenticación mediante tokens JWT, permitiendo mantener la sesión activa.

Capa 2: Lógica de negocio

Implementa el patrón MVC (Modelo-Vista-Controlador) y gestiona toda la lógica del sistema.

Sus componentes principales son:

Rutas: definen los endpoints REST y reciben las solicitudes del frontend.

Middlewares: validan la autenticación mediante tokens JWT y controlan la autorización de los usuarios.

Controladores: procesan las peticiones HTTP, aplican la lógica de negocio y coordinan la respuesta.

Modelos: se encargan de interactuar con la base de datos mediante consultas SQL.

Además, esta capa se integra con servicios externos:

Cloudinary: para subir y obtener imágenes de los perfiles.

WhatsApp: para facilitar el contacto directo entre usuarios y prestadores.

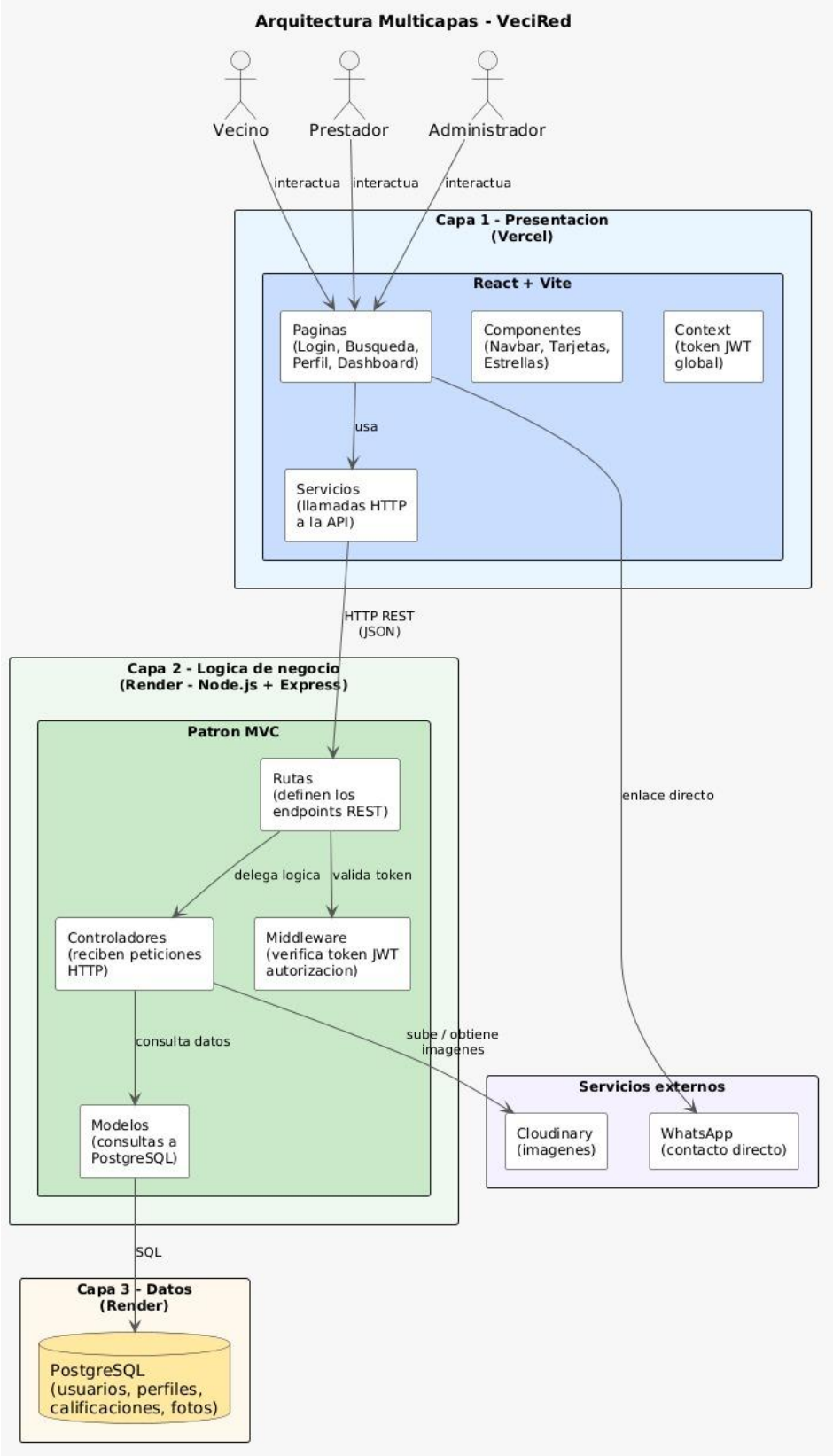
Capa 3: Datos (Base de datos)

Es la capa encargada del almacenamiento persistente de la información del sistema. Utiliza PostgreSQL para gestionar datos estructurados como:

- Usuarios
- Perfiles de prestadores
- Calificaciones
- Fotografías (URLs almacenadas)

Figura 8

Diagrama de capas



Nota: Elaboración propia

Tecnologías Que Se Utilizarán

Frontend (Capa de presentación)

- React: construcción de interfaces dinámicas y componentes reutilizables.
- Vite: herramienta de desarrollo rápida.
- HTML, CSS y JavaScript: estructura, estilos y lógica de la interfaz.
- Manejo de estado con Context API y autenticación con JWT.
- Vercel: despliegue del frontend.

Backend (Capa de lógica de negocio)

- Node.js: entorno de ejecución del servidor.
- Express: framework para crear la API REST.
- Arquitectura MVC: organización del código en rutas, controladores y modelos.
- Autenticación y autorización mediante JSON Web Tokens (JWT).
- Render: hosting del backend

Base de datos (Capa de datos)

- PostgreSQL: sistema de gestión de base de datos relacional para almacenar usuarios, perfiles, calificaciones y fotos.
- SQL: lenguaje para consultas y gestión de datos.
- Render: hosting de la base de datos.

Beneficios Esperados

Confianza en la comunidad: Los usuarios que deseen adquirir algún servicio van a poder verificar perfiles y revisar las reseñas de los prestadores locales.

Mejorar la economía hiperlocal: El capital circula dentro de cada comuna fortaleciendo la economía y fomentando las relaciones económicas sostenibles.

Reducción de barreras: Los trabajadores independientes pueden acceder al mercado digital y al cliente final sin intermediarios.

Eficiencia y agilidad: Los clientes van a poder encontrar de manera rápida y segura los servicios que necesiten.

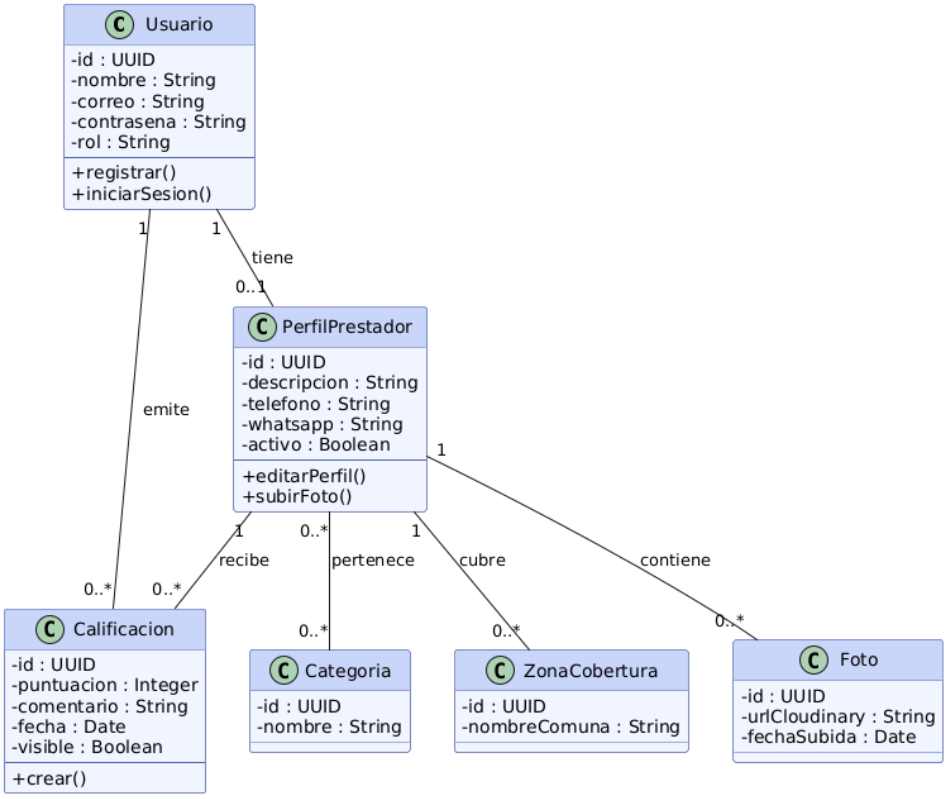
Modelamiento Del Sistema

Diagramas Estructurales

Diagrama De Clases

Figura 9

Diagrama de clases del proyecto VeciRed

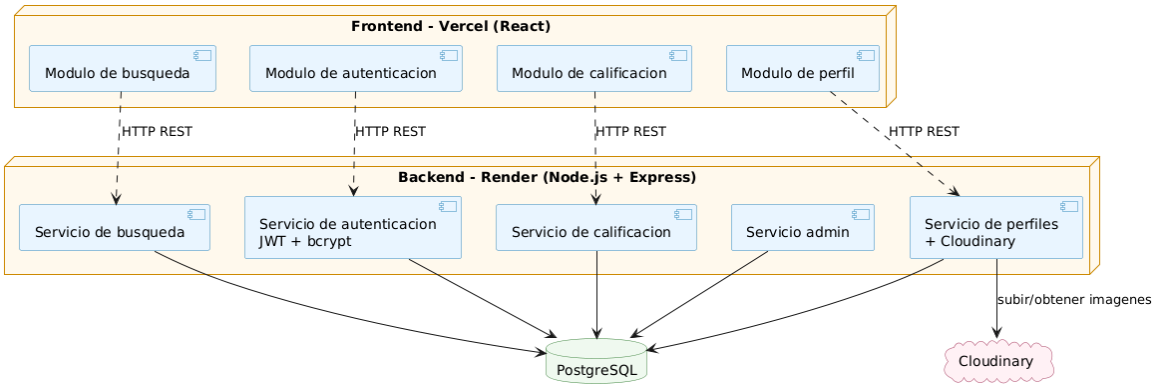


Nota: Elaboración Propia

Diagrama De Componentes

Figura 10

Diagrama de componentes del proyecto VeciRed



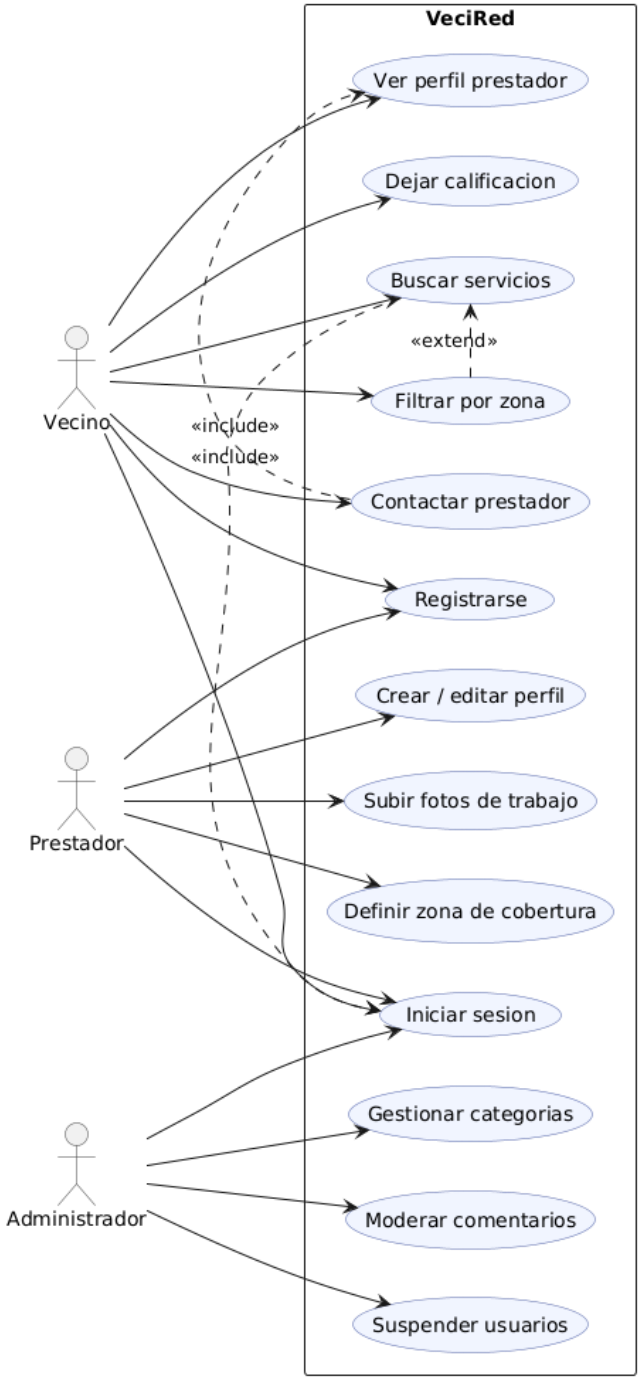
Nota: Elaboración Propia

Diagramas De Comportamiento

Diagrama De Casos De Uso

Figura 11

Diagrama de comportamiento del proyecto VeciRed

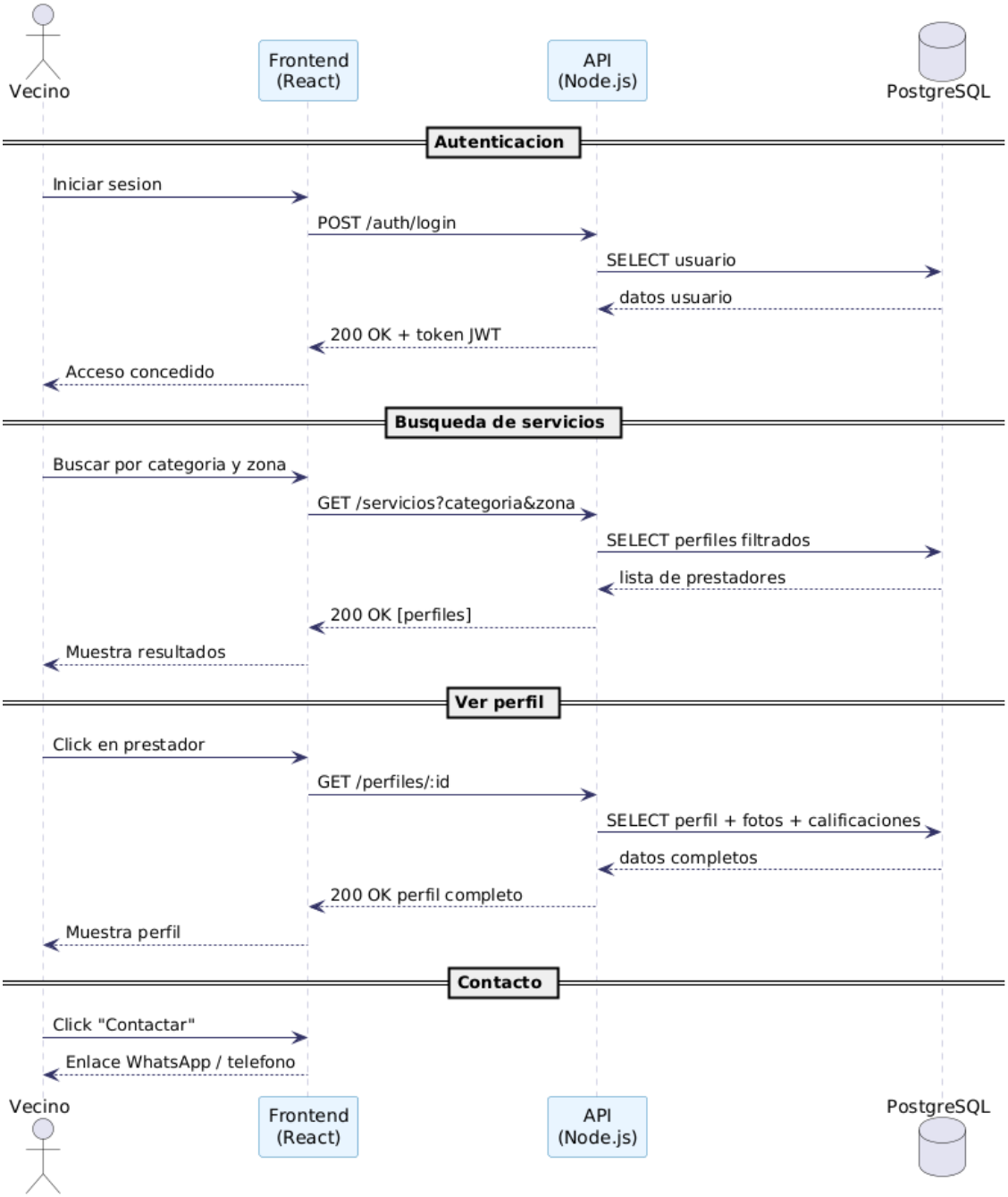


Nota: Elaboración Propia

Diagrama De Secuencia

Figura 12

Diagrama de secuencia del proyecto VeciRed

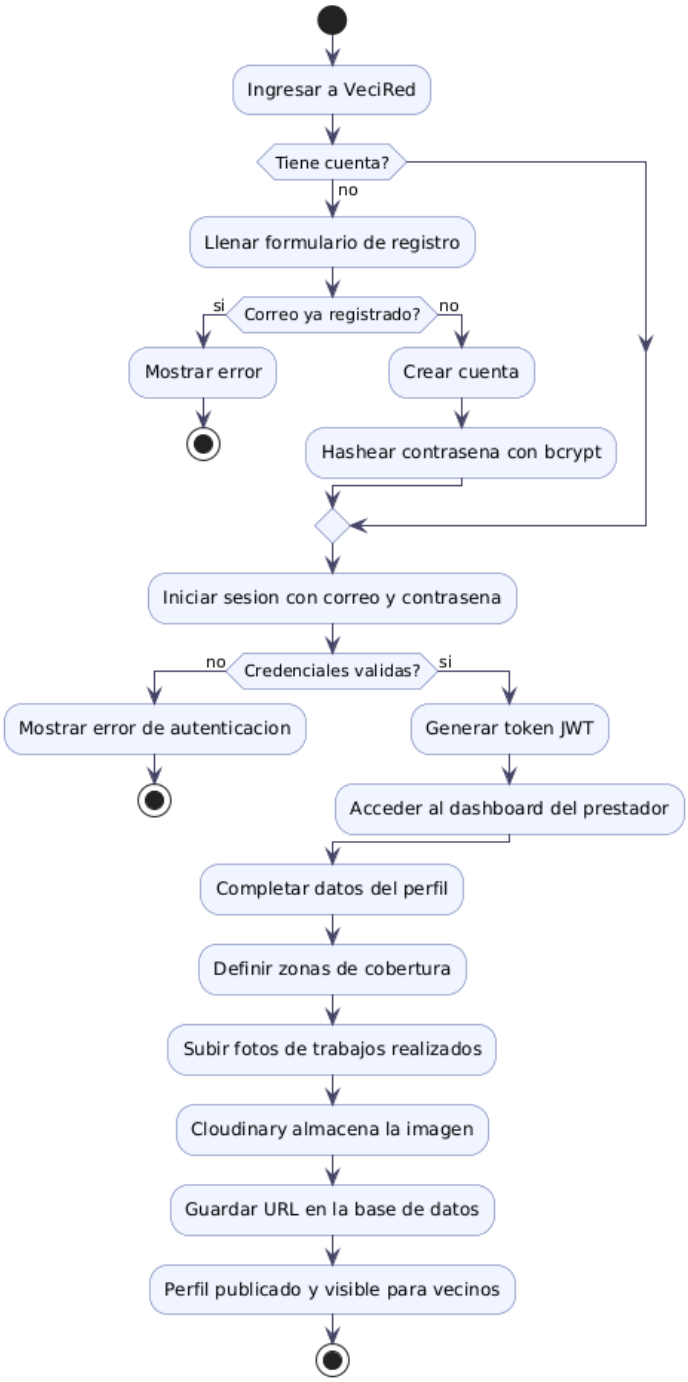


Nota: Elaboración Propia

Diagrama De Actividades

Figura 13

Diagrama de actividades del proyecto VeciRed



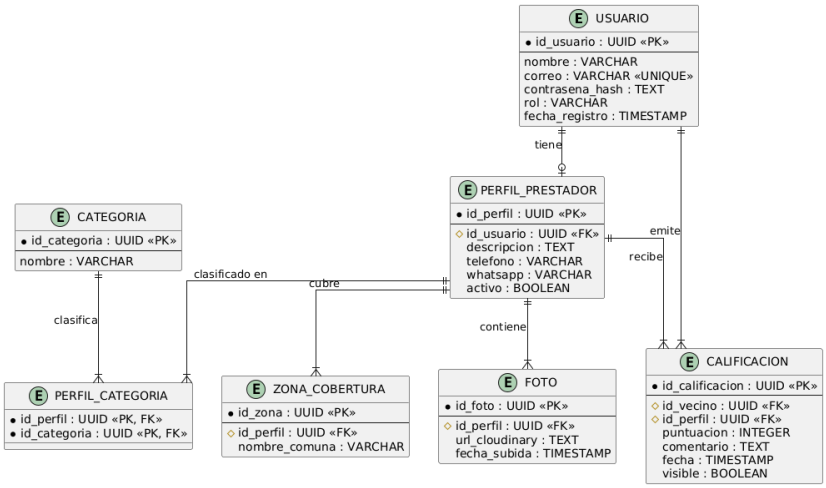
Nota: Elaboración Propia

Modelamiento De Base De Datos

Diagrama Entidad–Relación (ER)

Figura 14

Diagrama entidad – relación del proyecto VeciRed

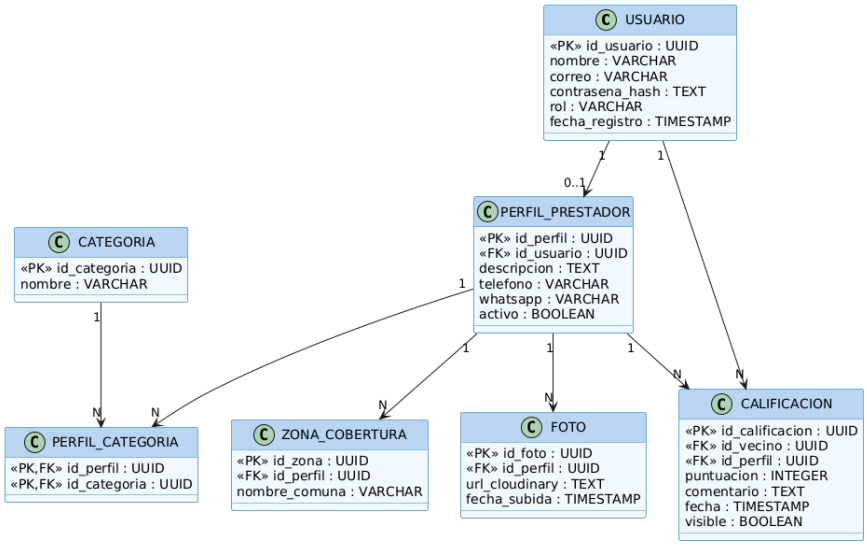


Nota: Elaboración Propia

Modelo Relacional

Figura 15

Diagrama del Modelo Relacional del proyecto VeciRed



Nota: Elaboración Propia

Segunda Entrega

VeciRed

Daisy Nicol Malagón Ramírez

Luz Celeste Rodríguez Cabarcas

Angela Marcela Salazar Peña

Juan Guillermo Sánchez Restrepo

Ingeniería En Software

Proyecto de Software

Corporación Universitaria Iberoamericana

Tatiana Lizbeth Cabrera Vargas

Bogotá D. C.

Abril, 2026

Actualización del MVP

Con base en la retroalimentación recibida durante la primera entrega, no se realizaron correcciones al MVP original, dado que el diseño, las historias de usuario y los requisitos funcionales fueron validados sin observaciones por parte de la docente. El equipo procedió directamente a la fase de implementación del prototipo funcional.

Durante el Sprint 1 se implementaron las dos funcionalidades core del sistema: el registro de usuarios y la autenticación con JWT. Estas funcionalidades constituyen la base del módulo de seguridad de VeciRed y son prerequisito para todas las demás funcionalidades planificadas en los sprints siguientes.

Estado del MVP por Historia de Usuario

Tabla 4

Estado del MVP por Historia de Usuario

ID	Historia	Actor	Estado
HU01	Como vecino, quiero registrarme en la plataforma	Vecino	**IMPLEMENTADA — Sprint 1**
HU02	Como vecino / prestador, quiero iniciar sesión en la plataforma	Vecino / Prestador	**IMPLEMENTADA — Sprint 1**
HU03	Como prestador, quiero crear y gestionar mi perfil	Prestador	Planificada — Sprint 2
HU04	Como vecino, quiero buscar servicios por categoría	Vecino	Planificada — Sprint 2
HU05	Como vecino, quiero filtrar resultados por ubicación	Vecino	Planificada — Sprint 2
HU06	Como vecino, quiero ver calificaciones y contactar al prestador	Vecino	Planificada — Sprint 2

Historias de Usuario Priorizadas

A continuación, se detallan las 6 historias de usuario priorizadas para el desarrollo de VeciRed, incluyendo descripción, criterios de aceptación y estado de implementación en el Sprint 1.

Tabla 5

HU01

HU01 — Como vecino, quiero registrarme en la plataforma para acceder a los servicios disponibles en mi comunidad	
Historia de Usuario	HU01 — Registrarme en la plataforma
Actor	Vecino
Funcionalidad	Registrarme en la plataforma
Beneficio	Acceder a los servicios disponibles en mi comunidad
Descripción	El vecino puede crear una cuenta en VeciRed ingresando su nombre, correo electrónico, contraseña y seleccionando su rol. El sistema valida que el correo no esté previamente registrado y almacena la contraseña cifrada con bcrypt.
Estado	IMPLEMENTADA — Sprint 1
Criterios de aceptación	• El usuario puede registrarse con nombre, correo y contraseña. • El sistema valida que el correo no esté previamente registrado. • La contraseña se almacena cifrada con bcrypt, nunca en texto plano. • El sistema redirige al login tras un registro exitoso. • Se muestra mensaje de error si el correo ya existe.

Tabla 6

HU02

HU02 — Como vecino / prestador, quiero iniciar sesión en la plataforma para acceder a mi cuenta y funcionalidades según mi rol	
Historia de Usuario	HU02 — Iniciar sesión en la plataforma
Actor	Vecino / Prestador
Funcionalidad	Iniciar sesión en la plataforma
Beneficio	Acceder a mi cuenta y funcionalidades según mi rol

Descripción	El usuario registrado puede autenticarse con su correo y contraseña. El sistema valida las credenciales, genera un token JWT y redirige al usuario según su rol (vecino al home, prestador al dashboard).
Estado	IMPLEMENTADA — Sprint 1
Criterios de aceptación	• El usuario puede iniciar sesión con correo y contraseña. • El sistema valida las credenciales contra la base de datos. • Se genera un token JWT con duración de 8 horas. • El token se almacena en el navegador para mantener la sesión. • Se muestra error si las credenciales son incorrectas.

Tabla 7

HU03

HU03 — Como prestador, quiero crear y gestionar mi perfil para dar a conocer mis servicios y generar confianza en los usuarios	
Historia de Usuario	HU03 — Crear y gestionar mi perfil
Actor	Prestador
Funcionalidad	Crear y gestionar mi perfil
Beneficio	Dar a conocer mis servicios y generar confianza en los usuarios
Descripción	El prestador puede completar su perfil con descripción del servicio, teléfono, WhatsApp, categorías de oficio y zonas de cobertura en las comunas de Medellín. También puede subir fotos de trabajos realizados.
Estado	Planificada — Sprint 2
Criterios de aceptación	• El prestador puede ingresar descripción, teléfono y WhatsApp. • Puede seleccionar una o más categorías de servicio. • Puede definir su zona de cobertura por comunas. • Puede subir fotos de trabajos realizados. • Puede actualizar su perfil en cualquier momento.

Tabla 8

HU04

HU04 — Como vecino, quiero buscar servicios por categoría para encontrar rápidamente el servicio que necesito	
Historia de Usuario	HU04 — Buscar servicios por categoría
Actor	Vecino
Funcionalidad	Buscar servicios por categoría
Beneficio	Encontrar rápidamente el servicio que necesito
Descripción	El vecino puede seleccionar una categoría de servicio (electricidad, plomería, pintura, etc.) y el sistema muestra una lista de prestadores disponibles que ofrecen ese servicio.
Estado	Planificada — Sprint 2
Criterios de aceptación	• El usuario puede seleccionar una categoría de servicio. • El sistema muestra prestadores relacionados con esa categoría. • Los resultados incluyen nombre, descripción y calificación del prestador. • La búsqueda arroja resultados correctos y relevantes.

Tabla 9

HU05

HU05 — Como vecino, quiero filtrar resultados por ubicación para identificar prestadores cercanos a mi zona	
Historia de Usuario	HU05 — Filtrar resultados por ubicación
Actor	Vecino
Funcionalidad	Filtrar resultados por ubicación
Beneficio	Identificar prestadores cercanos a mi zona
Descripción	El vecino puede aplicar un filtro por comuna o barrio para ver únicamente los prestadores que ofrecen servicios en su área geográfica dentro de las 16 comunas de Medellín.
Estado	Planificada — Sprint 2
Criterios de aceptación	• El usuario puede seleccionar una comuna como filtro. • El sistema muestra solo prestadores de esa ubicación. • El filtro se puede combinar con la búsqueda por categoría. • El filtro se aplica correctamente y en tiempo real.

Tabla 10

HU06

HU06 — Como vecino, quiero ver calificaciones y contactar al prestador para seleccionar un prestador confiable y coordinar el servicio	
Historia de Usuario	HU06 — Ver calificaciones y contactar al prestador
Actor	Vecino
Funcionalidad	Ver calificaciones y contactar al prestador
Beneficio	Seleccionar un prestador confiable y coordinar el servicio
Descripción	El vecino puede ver el perfil completo del prestador incluyendo su puntuación promedio, comentarios de otros vecinos y medios de contacto (teléfono y WhatsApp) para coordinar el servicio directamente.
Estado	Planificada — Sprint 2
Criterios de aceptación	• El sistema muestra la puntuación promedio del prestador. • Se visualizan comentarios verificados de otros vecinos. • La información es visible antes del contacto. • El sistema muestra teléfono y enlace directo a WhatsApp. • El usuario puede acceder fácilmente al contacto.

Prototipo funcional del sistema

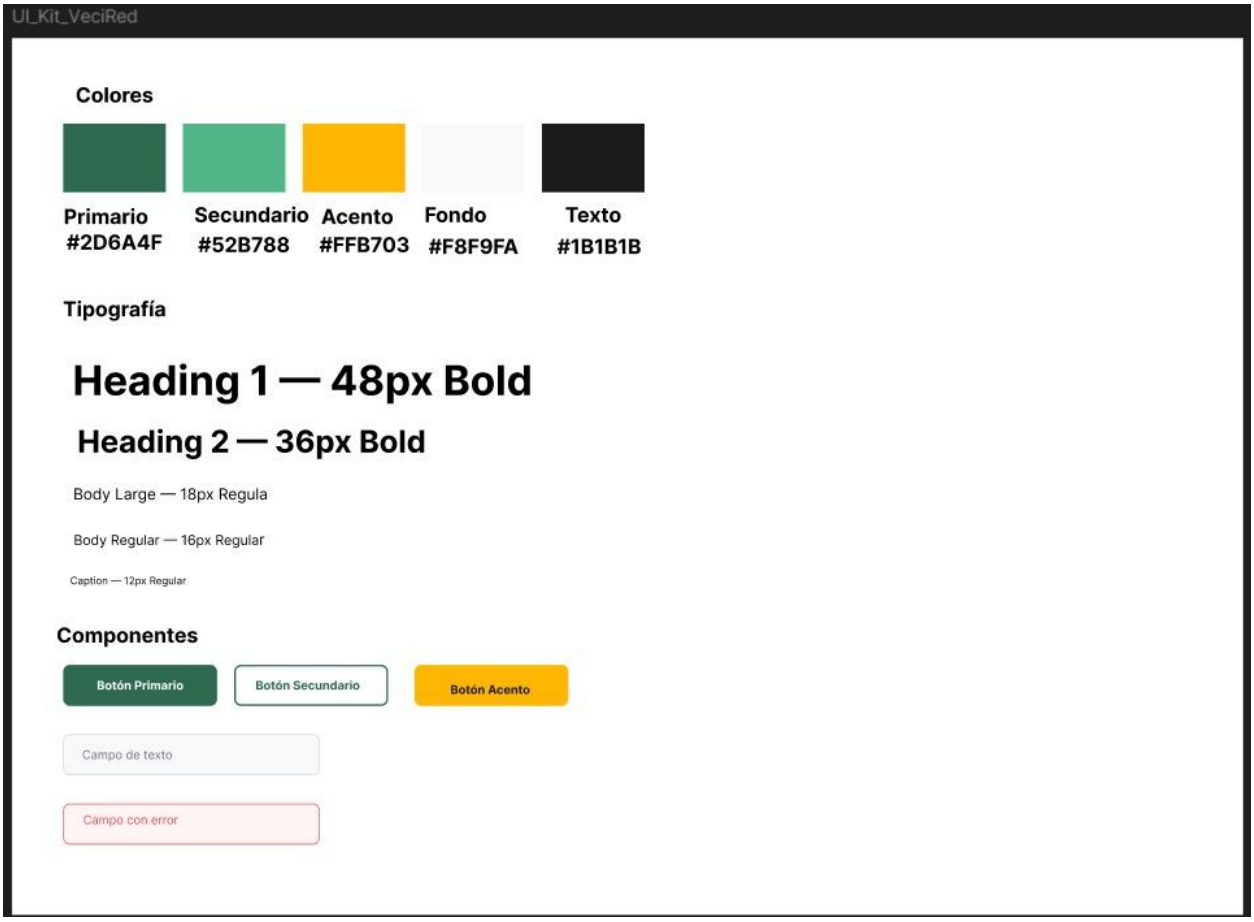
Prototipo de alta fidelidad

Link: https://www.figma.com/proto/PbVECiwapZmXqL8qGkcM6k/VeciRed-Prototipo-Alta-Fidelidad?node-id=1-1045&t=Ako7oFOC5pAOMwnK-1

UI Kit

Figura 16

Diseño del UI Kit



El UI Kit de VeciRed define los lineamientos visuales que garantizan la coherencia y consistencia de la interfaz en toda la plataforma.

Paleta de colores: El sistema utiliza cinco colores base. El verde oscuro #2D6A4F es el color primario, utilizado en botones principales, encabezados y elementos de navegación, y representa la confianza y lo comunitario. El verde medio #52B788 es el color secundario para elementos de apoyo y estados activos. El amarillo #FFB703 funciona como color de acento para llamadas a la acción destacadas. El blanco roto #F8F9FA es el color de fondo general de la aplicación, y el negro #1B1B1B se usa para el texto principal garantizando legibilidad.

Tipografía: El sistema define cinco niveles tipográficos. Heading 1 en 48px Bold para títulos principales, Heading 2 en 36px Bold para subtítulos de sección, Body Large en 18px Regular para texto destacado, Body Regular en 16px Regular para el contenido general, y Caption en 12px Regular para etiquetas y textos secundarios.

Componentes base: Se definieron tres variantes de botón — Botón Primario (verde oscuro, fondo sólido), Botón Secundario (contorno sin relleno) y Botón Acento (amarillo) — además de dos estados de campo de texto: estado normal y estado de error con borde rojo, para guiar al usuario en la validación de formularios.

Pruebas de usabilidad

El presente informe documenta los resultados de las pruebas de usabilidad realizadas al prototipo de alta fidelidad de VeciRed durante el Sprint 1. Las pruebas se llevaron a cabo utilizando la herramienta Maze, que permite medir el comportamiento real de los usuarios al interactuar con el prototipo de Figma.

Link: https://t.maze.co/530466435

Resumen de Resultados

Tabla 11

Prueba	**Participantes**	**Tiempo Promedio**	**Tasa de Éxito**
Regístrate en VeciRed	6 usuarios	4.9s	**100%**
Iniciar Sesión en VeciRed	5 usuarios	6.4s	**100%**
Identificar Categorías de Servicios	6 usuarios	7.8s	**100%**
Seleccionar Perfil Deseado	6 usuarios	5.8s	**100%**
Crear Cuenta Como Prestador	6 usuarios	5.6s	**100%**

Resultados Detallados por Prueba

Tabla 12

Prueba 1: Regístrate en VeciRed

Tarea asignada	Registrarse como vecino o como prestador en la plataforma
Participantes	6 usuarios
Tiempo promedio	4.9 segundos
Tasa de éxito	**100% (6/6 usuarios completaron la tarea)**

Análisis	La mayoría de los usuarios completó el flujo de registro de forma directa. Un usuario exploró pantallas adicionales antes de completar la tarea, lo que indica curiosidad por conocer más del prototipo. El tiempo promedio es bajo, indicando que el formulario es intuitivo.

Figura 17

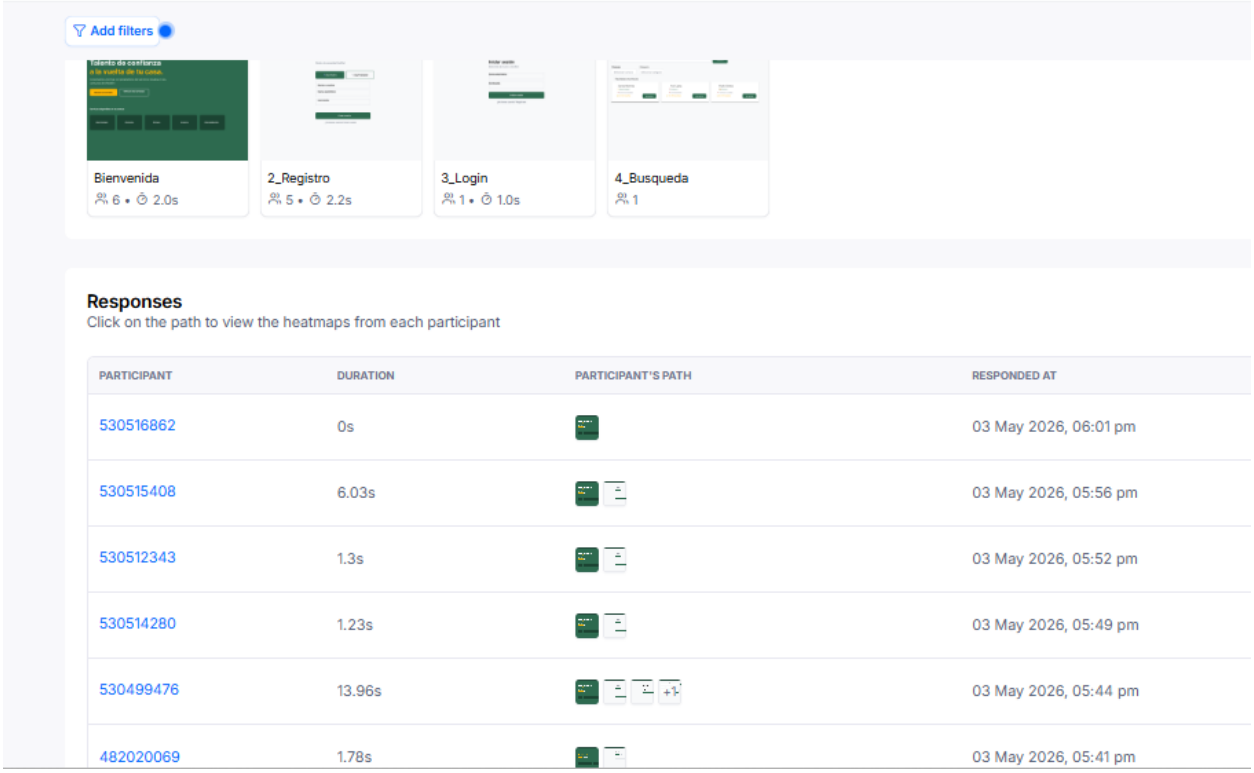


Tabla 13

Prueba 2: Iniciar Sesión en VeciRed

Tarea asignada	Iniciar sesión con credenciales proporcionadas
Participantes	5 usuarios — 3 de mayo de 2026
Tiempo promedio	6.4 segundos
Tasa de éxito	100% (5/5 usuarios completaron la tarea)
Análisis	El flujo de inicio de sesión fue completado exitosamente por todos los participantes. Un usuario tomó más tiempo explorando las pantallas disponibles. El tiempo de 0s de un participante indica que ya conocía el flujo de navegación.

Figura

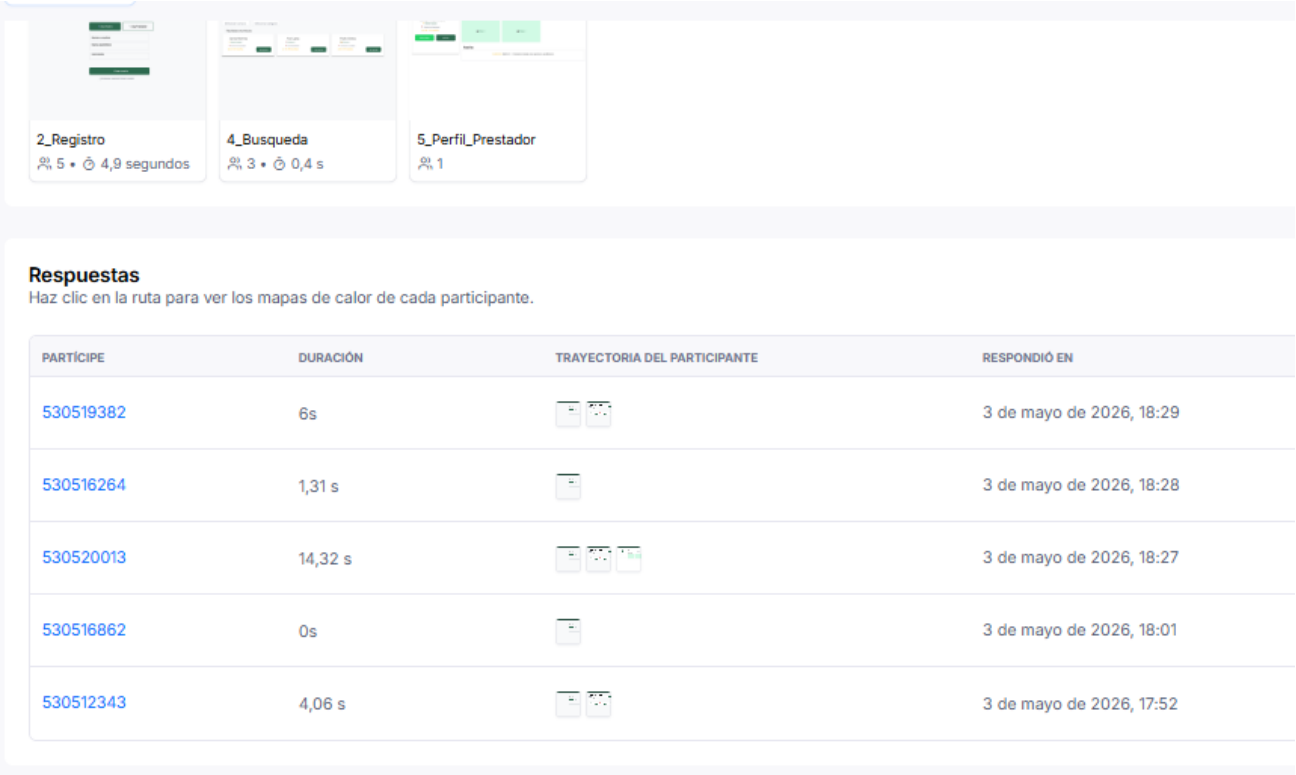


Tabla 14

Prueba 3: Identificar Categorías de Servicios

Tarea asignada	Identificar qué categorías de servicios ofrece VeciRed
Participantes	6 usuarios
Tiempo promedio	7.8 segundos
Tasa de éxito	100% (6/6 usuarios completaron la tarea)
Análisis	Todos los participantes lograron identificar las categorías de servicios. La mayoría navegó hacia la pantalla de búsqueda y perfil del prestador, lo que confirma que el flujo de descubrimiento es natural e intuitivo. Los tiempos más altos corresponden a usuarios que exploraron el perfil del prestador con detalle.

Figura

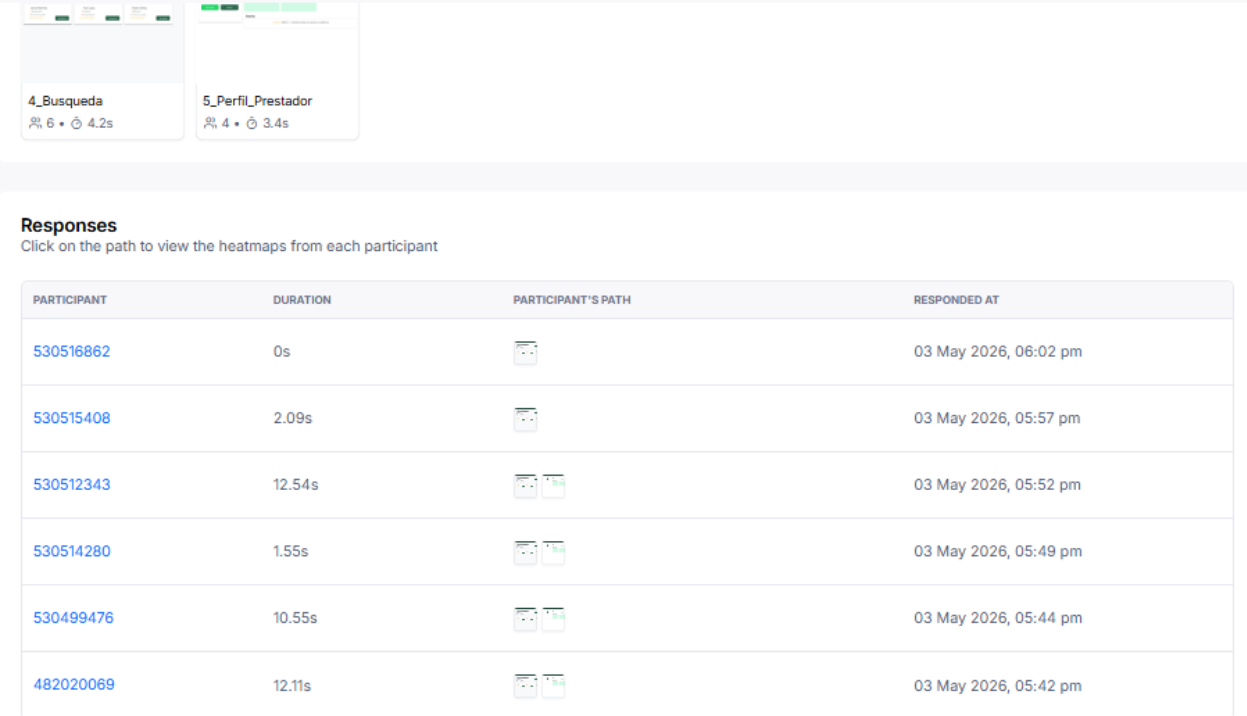


Tabla 15

Prueba 4: Seleccionar Perfil Deseado

Tarea asignada	Seleccionar un perfil de prestador de la lista de resultados
Participantes	6 usuarios
Tiempo promedio	5.8 segundos
Tasa de éxito	**100% (6/6 usuarios completaron la tarea)**
Análisis	La mayoría completó la tarea en menos de 3 segundos, lo que indica que la selección de perfiles es muy intuitiva. Un participante tardó posiblemente comparando múltiples opciones antes de decidir, comportamiento esperado y positivo en una plataforma de servicios.

Figura

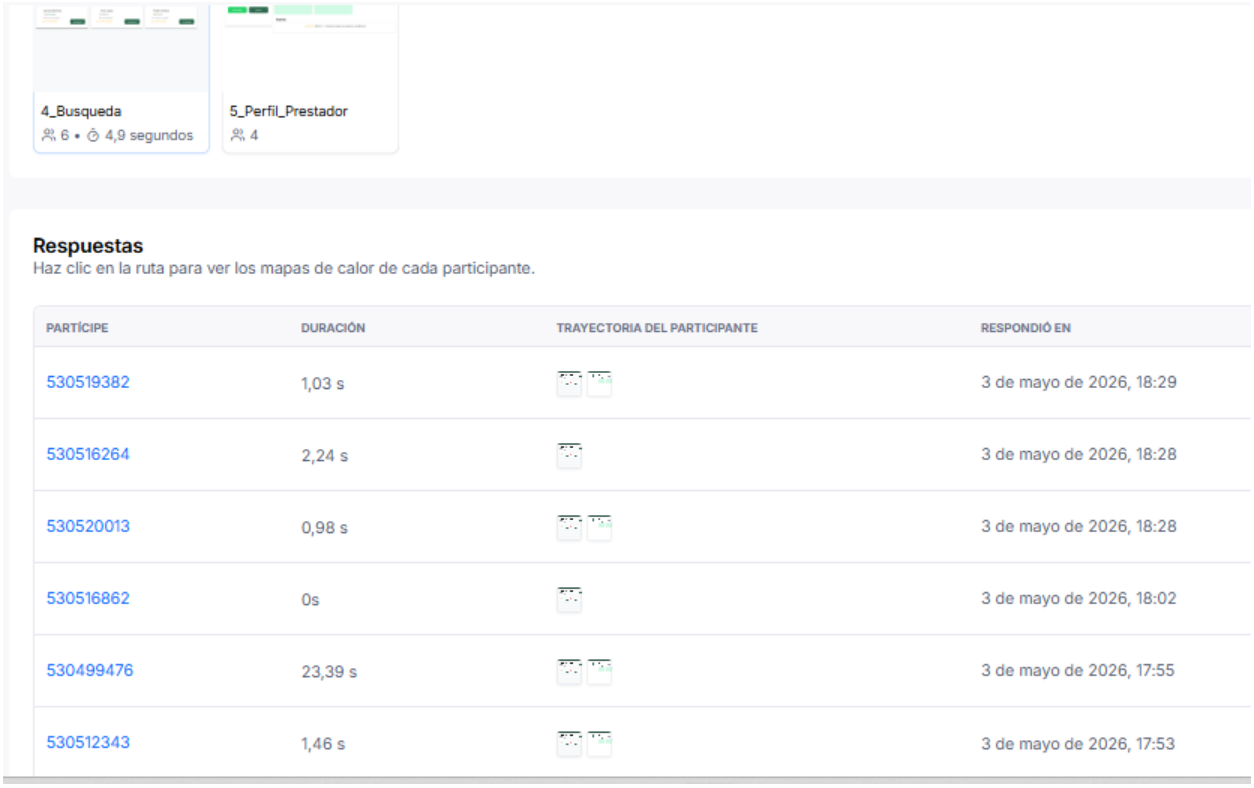


Tabla 16

Prueba 5: Crear Cuenta Como Prestador

Tarea asignada	Crear una cuenta seleccionando el rol de prestador de servicios
Participantes	6 usuarios — 3 de mayo de 2026
Tiempo promedio	5.6 segundos
Tasa de éxito	100% (6/6 usuarios completaron la tarea)
Análisis	Tres participantes completaron la tarea rápidamente. Los tres indican que navegaron directamente sin incertidumbre, demostrando que el flujo de registro como prestador es claro y diferenciado del registro como vecino.

Figura

5_Perfil_Prestador
6 • 2,8 s

Respuestas
Haz clic en la ruta para ver los mapas de calor de cada participante.

PARTÍCIPE	DURACIÓN	TRAYECTORIA DEL PARTICIPANTE	RESPONDIÓ EN
530519382	2,12 s		3 de mayo de 2026, 18:29
530516264	3,09 s		3 de mayo de 2026, 18:28
530520013	11,65 s		3 de mayo de 2026, 18:28
530516862	0s		3 de mayo de 2026, 18:02
530499476	0s		3 de mayo de 2026, 17:55
530512343	0s		3 de mayo de 2026, 17:53

Los resultados de las pruebas de usabilidad del Sprint 1 de VeciRed demuestran que el prototipo de alta fidelidad tiene una curva de aprendizaje baja y flujos de navegación intuitivos. Las 5 pruebas realizadas obtuvieron una tasa de éxito del 100%, lo que confirma que los usuarios pueden completar las tareas principales sin necesidad de instrucciones adicionales.

Hallazgos positivos

El flujo de registro como vecino y como prestador es claro y diferenciado, los usuarios lo completaron en tiempos muy bajos.

La pantalla de inicio de sesión es reconocible e intuitiva, todos los participantes identificaron correctamente los campos de correo y contraseña.

Las categorías de servicios en el home son fácilmente identificables, confirmando que el diseño visual comunica correctamente la propuesta de valor de la plataforma.

La navegación entre pantallas es fluida y los usuarios exploraron naturalmente el perfil del prestador, comportamiento esperado en una plataforma de servicios.

Oportunidades de mejora para Sprint 2

Reducir los tiempos de exploración en la selección de perfiles mediante la implementación de filtros por categoría y zona geográfica (planificado en Sprint 2).

Agregar indicadores visuales más claros en el formulario de registro para diferenciar mejor el rol de vecino y el rol de prestador.

Implementar mensajes de confirmación más visibles tras completar acciones importantes como el registro exitoso.

Arquitectura del sistema

Diagramas C4

Los diagramas C4 permiten describir la arquitectura del sistema en niveles de abstracción progresivos. Para VeciRed se presentan el Nivel 1 (Contexto) y el Nivel 2 (Contenedores), siguiendo el modelo propuesto por Simon Brown.

Nivel 1 (Contexto)

El diagrama de contexto muestra a VeciRed como sistema central y sus relaciones con los actores externos: los tres tipos de usuarios (Vecino, Prestador y Administrador) y los servicios externos (Cloudinary, WhatsApp, Vercel y Render).

Descripción de actores:

Vecino: habitante de la comuna que busca, filtra y contacta prestadores de servicios confiables.

Prestador: trabajador independiente que crea su perfil digital, sube fotos de trabajos y recibe calificaciones.

Administrador: gestiona usuarios, modera comentarios y administra el catálogo de categorías.

Cloudinary: servicio externo de almacenamiento y optimización de imágenes.

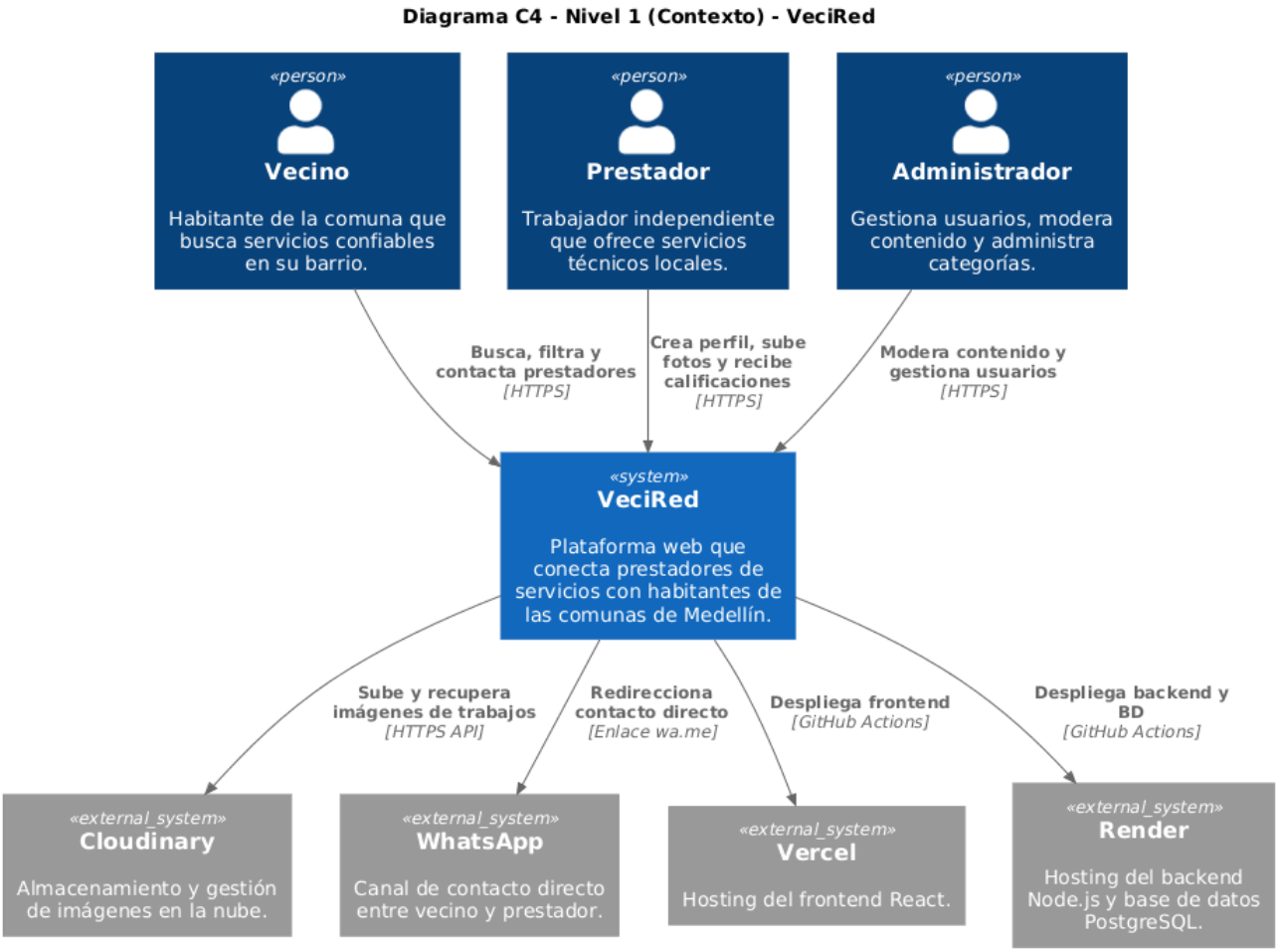
WhatsApp: canal de contacto directo entre vecino y prestador fuera de la plataforma.

Vercel: plataforma de hosting del frontend React con deploy automático.

Render: plataforma de hosting del backend Node.js y la base de datos PostgreSQL.

Figura

Diagrama C4 – Nivel 1 (Contexto)



Nivel 2 (Contenedores)

El diagrama de contenedores desglosa el interior de VeciRed en sus tres contenedores principales: el frontend React, el backend Node.js + Express y la base de datos PostgreSQL. Muestra cómo se comunican entre sí y con los servicios externos.

Descripción de contenedores:

Frontend React + Vite (Vercel): interfaz de usuario con páginas de Login, Registro, Home, Búsqueda, Perfil y Dashboard. Se comunica con el backend mediante HTTP REST con JSON.

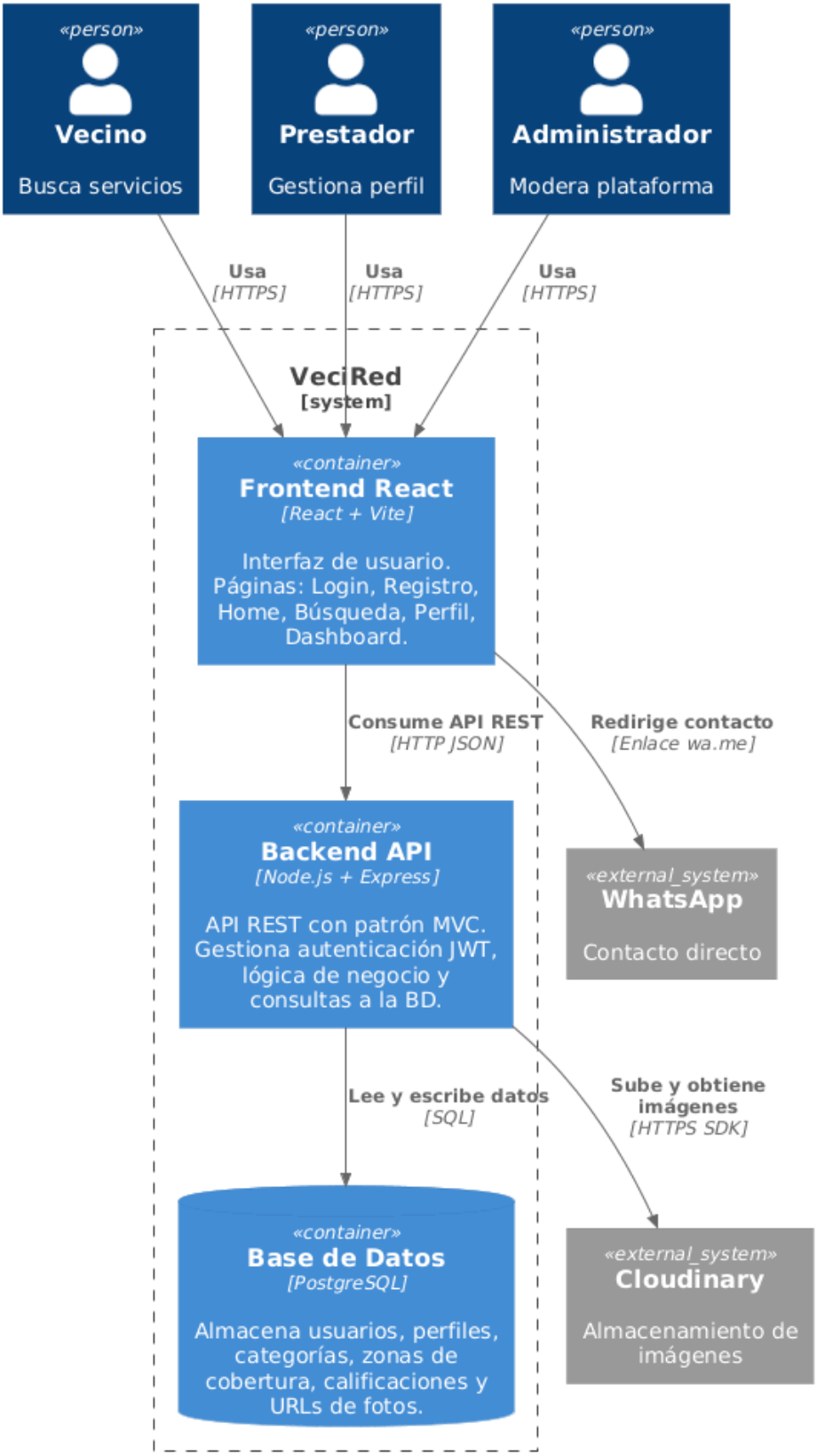
Backend API Node.js + Express (Render): implementa el patrón MVC. Gestiona la autenticación JWT, aplica la lógica de negocio y realiza consultas SQL a PostgreSQL. Se integra con Cloudinary para imágenes.

Base de Datos PostgreSQL (Render): almacena de forma persistente usuarios, perfiles de prestadores, categorías, zonas de cobertura, calificaciones y URLs de fotos de Cloudinary.

Figura

Diagrama C4 – Nivel 2 (Contenedores)

Diagrama C4 - Nivel 2 (Contenedores) - VeciRed



Documento ADR

A continuación seregistra las Decisiones de Arquitectura tomadas durante el Sprint 1 del desarrollo de VeciRed. Cada decisión incluye el contexto del problema, las alternativas evaluadas y la justificación de la elección realizada.

VeciRed es una plataforma web orientada a conectar prestadores de servicios técnicos informales con habitantes de las comunas de Medellín, bajo una arquitectura de tres capas con patrón MVC en el backend.

Registro de Decisiones de Arquitectura

Tabla

Decisiones técnicas

Decisión	Elección	Justificación
Lenguaje de programación backend	JavaScript (Node.js)	Permite usar el mismo lenguaje en frontend y backend. Ecosistema npm amplio, manejo nativo de operaciones asíncronas y gran comunidad de soporte.
Framework backend	Express.js	Framework minimalista y flexible para Node.js. Facilita la creación de APIs REST con poco código, amplia documentación y compatibilidad con múltiples middlewares.
Lenguaje de programación frontend	JavaScript + JSX	Estándar de la industria para React. Permite escribir componentes de interfaz con sintaxis declarativa y reutilizable.
Framework frontend	React + Vite	React permite construir interfaces dinámicas con componentes reutilizables. Vite ofrece un servidor de desarrollo más rápido que Create React App, mejorando la experiencia de desarrollo.
Base de datos	PostgreSQL	Sistema relacional robusto, ACID-compliant, gratuito y de código abierto. Compatible con

		el plan gratuito de Render y soporta tipos de datos avanzados como UUID nativo.
Autenticación	JWT (JSON Web Tokens)	Permite autenticación stateless, sin necesidad de mantener sesiones en el servidor. Estándar para APIs REST, compatible con cualquier cliente y fácil de implementar con la librería jsonwebtoken.
Cifrado de contraseñas	bcryptjs	Implementación segura del algoritmo bcrypt para hashear contraseñas. Incluye salt automático y es resistente a ataques de fuerza bruta.
Tipo de arquitectura	Cliente-servidor + API REST	Permite separación clara entre frontend y backend. El frontend consume la API mediante HTTP, lo que facilita el escalado independiente de cada capa y la reutilización de la API en futuros clientes móviles.
Hosting frontend	Vercel	Plataforma especializada en React con deploy automático desde GitHub. Ofrece CDN global, HTTPS automático y capa gratuita suficiente para el proyecto.
Hosting backend y BD	Render	Soporta Node.js y PostgreSQL en su capa gratuita. Integración con GitHub para despliegue continuo y configuración de variables de entorno segura.
Gestión de imágenes	Cloudinary	Servicio especializado en almacenamiento y transformación de imágenes. SDK gratuito, URLs permanentes para las fotos de los prestadores y optimización automática de formatos.
Control de versiones	Git + GitHub	Estándar de la industria. Permite trabajo colaborativo entre los 4 integrantes, gestión de ramas con Git Flow y trazabilidad completa del historial del proyecto.

Decisiones Detalladas

ADR-001: Arquitectura General del Sistema

Estado: Aceptada - Fecha: Mayo 2026

El sistema necesita separar claramente la interfaz de usuario, la lógica de negocio y la persistencia de datos, permitiendo que cada capa evolucione de forma independiente.

Decisión:

Se adopta una arquitectura de tres capas (3-tier): Capa de Presentación (React + Vite en Vercel), Capa de Lógica de Negocio (Node.js + Express en Render con patrón MVC) y Capa de Datos (PostgreSQL en Render).

Consecuencias:

Positiva: Separación clara de responsabilidades y escalabilidad independiente por capa.

Positiva: El frontend puede consumir la API desde cualquier cliente futuro (móvil, etc.).

Negativa: Mayor complejidad inicial comparado con una arquitectura monolítica.

ADR-002: Lenguaje y Framework Backend

Estado: Aceptada - Fecha: Mayo 2026

Se requiere un servidor HTTP capaz de manejar múltiples peticiones concurrentes con baja latencia, que pueda integrarse con PostgreSQL y generar tokens JWT.

Alternativas evaluadas:

- Python + FastAPI: excelente rendimiento pero lenguaje diferente al frontend.
- Java + Spring Boot: robusto pero excesivo para el alcance del proyecto y curva de aprendizaje alta.
- Node.js + Express: mismo lenguaje que el frontend, ecosistema npm, manejo nativo de async/await.

Decisión:

Node.js con Express.js. El equipo tiene experiencia previa y permite unificar el lenguaje de programación en todo el stack.

ADR-003: Base de Datos Relacional

Estado: Aceptada - Fecha: Mayo 2026

La plataforma maneja datos estructurados con relaciones entre usuarios, perfiles, categorías, calificaciones y fotos. Se requiere integridad referencial y soporte de transacciones.

Alternativas evaluadas:

- MongoDB: flexible pero sin integridad referencial nativa, menos adecuado para datos relacionales.
- MySQL: relacional y gratuito, pero menos soporte en Render y sin UUID nativo.
- PostgreSQL: ACID-compliant, UUID nativo con gen_random_uuid(), soportado en Render Free Tier.

Decisión:

PostgreSQL. Garantiza integridad de datos, soporta el modelo relacional definido y es compatible con el hosting gratuito seleccionado.

ADR-004: Estrategia de Autenticación

Estado: Aceptada - Fecha: Mayo 2026

El sistema requiere autenticación segura con roles diferenciados (vecino, prestador, administrador) y protección de endpoints sensibles.

Alternativas evaluadas:

- Sesiones con cookies: requiere estado en el servidor, más complejo de escalar.
- OAuth2 con Google: simplifica el registro pero agrega dependencia externa y complejidad.
- JWT (JSON Web Tokens): stateless, estándar REST, el token viaja en cada petición sin consultar BD.

Decisión:

JWT con bcryptjs para el hash de contraseñas. El token se firma con una clave secreta y expira en 8 horas. Las contraseñas se almacenan siempre hasheadas, nunca en texto plano.

ADR-005: Gestión de Imágenes

Estado: Aceptada - Fecha: Mayo 2026

Los prestadores deben poder subir fotos de sus trabajos.

Decisión:

Cloudinary. Ofrece SDK gratuito, almacenamiento en la nube, URLs permanentes y transformaciones automáticas de imágenes. Solo se almacena la URL en PostgreSQL.

ADR-006: Estrategia de Despliegue

Estado: Aceptada | Fecha: Mayo 2026

Decisión:

- Frontend: Vercel — deploy automático desde GitHub, CDN global, HTTPS automático, capa gratuita.
- Backend y Base de Datos: Render — soporta Node.js y PostgreSQL en Free Tier, variables de entorno seguras, deploy desde GitHub.
- Control de versiones: Git Flow con GitHub — rama main para producción, develop para integración, feature branches para cada funcionalidad.

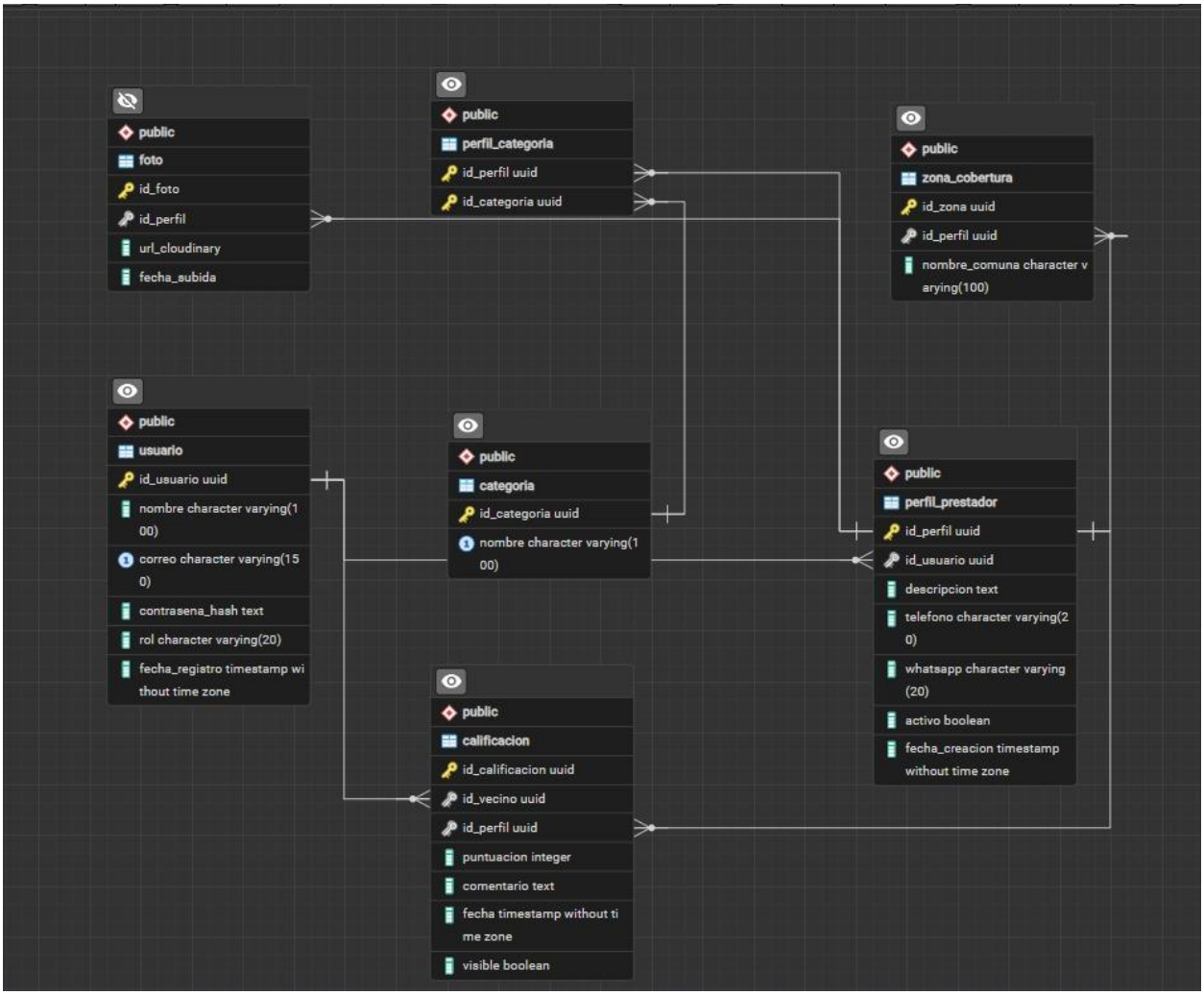
Repositorio del proyecto

Link https://github.com/Celeste-tech/VeciRed

Modelamiento de Base de Datos — Diagrama Entidad Relación (ERD)

Figura

Diseño del backend



El diagrama presenta el modelo relacional implementado en PostgreSQL para la plataforma VeciRed. El sistema está compuesto por siete tablas interrelacionadas que soportan todas las funcionalidades del sistema.

La tabla central es USUARIO, que almacena los datos de todos los actores del sistema (vecinos, prestadores y administradores) con sus atributos de nombre, correo, contraseña hasheada, rol y fecha de registro. A partir de esta tabla se desprende PERFIL_PRESTADOR, que extiende la información del usuario con rol prestador, incluyendo descripción del servicio, teléfono, WhatsApp y estado activo.

El perfil del prestador se relaciona con tres tablas adicionales. PERFIL_CATEGORIA es una tabla intermedia de relación N:N que vincula cada perfil con una o más categorías de

servicio definidas en la tabla CATEGORIA (electricidad, plomería, pintura, etc.). La tabla ZONA_COBERTURA registra las comunas de Medellín donde el prestador ofrece sus servicios. La tabla FOTO almacena las URLs de Cloudinary de las imágenes de trabajos realizados por el prestador.

Finalmente, la tabla CALIFICACION registra las valoraciones que los vecinos hacen a los prestadores, con puntuación del 1 al 5, comentario opcional, fecha y visibilidad. Esta tabla se relaciona tanto con USUARIO (quien califica) como con PERFIL_PRESTADOR (quien es calificado), con una restricción UNIQUE que garantiza que un vecino solo pueda calificar una vez a cada prestador.

Retrospectiva del Sprint

¿Qué funcionó bien?

Definición clara de la arquitectura del sistema desde el inicio del Sprint: la decisión de adoptar una arquitectura de tres capas con patrón MVC en el backend permitió que cada integrante entendiera su rol técnico y avanzara sin bloqueos.

Implementación exitosa de las dos funcionalidades core del Sprint: registro de usuarios y autenticación con JWT, ambas probadas con Postman y conectadas a la base de datos PostgreSQL real.

Configuración completa del stack tecnológico en entorno local: Node.js, Express, PostgreSQL (pgAdmin), React + Vite y Postman funcionando de forma integrada.

¿Qué dificultades se presentaron?

Problemas con la lectura de variables de entorno desde el archivo. env: la librería dotenv no estaba leyendo correctamente las variables debido a una interferencia con dotenvx instalado globalmente, lo que causó que DB_PASSWORD y JWT_SECRET llegaran como undefined al servidor.

Como solución temporal se hardcodearon las credenciales directamente en los archivos db.js y auth.controller.js para continuar el desarrollo. Esta deuda técnica debe resolverse antes de subir a producción.

¿Qué mejoraremos en el siguiente Sprint?

Estandarizar la configuración de variables de entorno desde el inicio del Sprint: crear un archivo .env.example en el repositorio con todas las variables necesarias (sin valores reales) para que cada integrante configure su entorno local de forma consistente antes de comenzar a desarrollar.

Resolver la deuda técnica del .env antes de subir el código a producción en Render, usando correctamente las variables de entorno del panel de Render en lugar de valores hardcodeados.

Establecer un checklist de configuración inicial de entorno que todo integrante debe completar al inicio de cada Sprint, evitando pérdida de tiempo en problemas de configuración local.

Implementar ESLint y Prettier en el repositorio para mantener consistencia en el estilo de código entre todos los integrantes del equipo.

Firmas del Equipo

Los integrantes del equipo VeciRed confirman haber participado activamente en el Sprint 1 y estar de acuerdo con el contenido.

Daisy Nicol Malagón Ramírez	**Luz Celeste Rodríguez Cabarcas**

Angela Marcela Salazar Peña	**Juan Guillermo Sánchez Restrepo**

Video

Link: https://youtu.be/gQnZ6ckFuis

Conclusiones

El desarrollo de la plataforma VeciRed permite evidenciar la aplicación integral de conceptos de análisis, diseño y modelamiento de sistemas orientados a resolver problemáticas reales mediante soluciones tecnológicas. A partir de la estructuración de flujos de usuario, diagramas de proceso y definición de arquitectura, se logró establecer una base sólida para la implementación de un sistema enfocado en la conexión de servicios dentro de economías hiperlocales.

La implementación de la metodología ágil Scrum en el desarrollo de VeciRed permitió estructurar el proyecto de manera iterativa e incremental, facilitando la entrega continua de valor y la adaptación a cambios durante el proceso. A través de la organización en Sprints, fue posible dividir el desarrollo en etapas manejables, priorizando funcionalidades clave.

El modelamiento de flujos de interacción y diagramas de flujo permitió identificar de manera anticipada los distintos escenarios de uso, incluyendo validaciones, manejo de errores y toma de decisiones, lo cual contribuye a mejorar la experiencia del usuario y reducir fallos en la implementación. El uso de tecnologías modernas como React y Node.js garantiza un desarrollo ágil y eficiente, además de una experiencia de usuario dinámica e interactiva.

Las decisiones de arquitectura tomadas priorizan la viabilidad técnica dentro de las restricciones de presupuesto cero y plazo académico, sin sacrificar las buenas prácticas de ingeniería de software. La combinación Node.js + Express + React + PostgreSQL forma un stack cohesivo, bien documentado y con amplio soporte comunitario. La arquitectura de tres capas con patrón MVC en el backend garantiza la separación de responsabilidades, facilitando el mantenimiento y la incorporación de nuevas funcionalidades en los sprints siguientes.

Referencias

Alcaldía de Medellín. (2025). *Informe del mercado laboral y observatorio de desarrollo económico*. Secretaría de Desarrollo Económico. https://www.medellin.gov.co

Cámara de Comercio de Medellín para Antioquia. (2025). *Estudio sobre micronegocios y digitalización en barrios periféricos del Valle de Aburrá*. https://www.camaramedellin.com.co

Comunas de Medellín. (2026, 3 de abril). En *Wikipedia, la enciclopedia libre*. https://es.wikipedia.org/wiki/Comunas_de_Medellín

Departamento Administrativo Nacional de Estadística [DANE]. (2026). *Boletín técnico: Medición de empleo informal y seguridad social (Trimestre noviembre 2025 - enero 2026)*. https://www.dane.gov.co

El colombiano. (2025, 15 de diciembre). *Análisis del mercado laboral en el Valle de Aburrá: Retos de la informalidad*. https://www.elcolombiano.com

Medellín Cómo Vamos. (2025). *Informe de calidad de vida de Medellín: Mercado laboral, pobreza y desigualdad*. https://www.medellincomovamos.org

Ministerio de Tecnologías de la Información y las Comunicaciones [MinTIC]. (2025). *Estudio nacional de brecha digital y talento TI 2025-2030*. https://www.mintic.gov.co

Schwaber, K., y Sutherland, J. (2020). *La guía de Scrum: Las reglas del juego*. Scrum.org. https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-Latin-American.pdf

Google. (2026). *Matriz de stakeholders para el proyecto VeciRed* [Imagen generada por IA]. Gemini 3 Flash. https://gemini.google.com/

Sánchez, C. (08 de febrero de 2019). Normas APA – 7ma (séptima) edición. Normas APA (7ma edición). https://normas-apa.org/

Anthropic. (2026). Claude 3.5 Sonnet (Versión de mayo) [Modelo de lenguaje grande]. https://www.anthropic.com/claude

Figma Inc. (2026). Figma (Versión 124.5.5) [Software de diseño gráfico basado en la nube]. https://www.figma.com

Maze Design Inc. (2026). Maze [Plataforma de pruebas de usuario y prototipado]. https://maze.co

Microsoft. (2026). Visual Studio Code (Versión 1.90) [Software de computación]. https://code.visualstudio.com/

OpenJS Foundation. (2026). Node.js (Versión 20.12.2) [Entorno de ejecución de JavaScript]. https://nodejs.org/

Express.js. (2026). Express (Versión 4.19.2) [Marco de trabajo para aplicaciones web de Node.js]. https://expressjs.com/

PostgreSQL Global Development Group. (2026). PostgreSQL (Versión 16.2) [Sistema de gestión de bases de datos relacionales]. https://www.postgresql.org/

Auth0. (2026). JSON Web Token [Estándar de seguridad]. https://jwt.io/ Kelektiv. (2026). bcryptjs (Versión 2.4.3) [Librería de funciones de hash para JavaScript]. https://www.npmjs.com/package/bcryptjs